

CLOVER: Closed-Loop Value Estimation & Ranking for End-to-End Autonomous Driving Planning

Sining Ang^{1,2} Yuguang Yang^{3,2} Canyu Chen^{4,2} Yan Wang^{2*}

¹Department of Automation, University of Science and Technology of China

²Institute for AI Industry Research, Tsinghua University

³School of Electronic Information Engineering, Beihang University

⁴National College for Excellent Engineers, Beihang University

angsn@mail.ustc.edu.cn wangyan@air.tsinghua.edu.cn

Abstract

End-to-end autonomous driving planners are commonly trained by imitating a single logged trajectory, yet they are evaluated by rule-based planning metrics that measure safety, feasibility, progress, and comfort. This creates a training–evaluation mismatch: trajectories close to the logged path may still violate planning rules, while alternative trajectories farther from the demonstration can remain valid and high-scoring. The mismatch is especially limiting for proposal-selection planners, whose performance depends on both candidate-set coverage and scorer ranking quality. We propose **CLOVER**, a **C**losed-**L**oop **V**alue **E**stimation and **R**anking framework for end-to-end autonomous driving planning. CLOVER follows a lightweight generator–scorer formulation: a generator produces diverse candidate trajectories, and a trajectory-level scorer predicts planning-metric sub-scores to rank them at inference time. To expand proposal support beyond single-trajectory imitation, CLOVER constructs evaluator-filtered pseudo-expert trajectories and trains the generator with set-level coverage supervision. It then performs conservative closed-loop self-distillation: the scorer is fitted to true evaluator sub-scores on generated proposals, while the generator is refined toward teacher-selected top- k and vector-Pareto proposal targets with stability regularization. We further analyze when an imperfect scorer can improve the generator, showing that scorer-mediated refinement is reliable when scorer-selected targets are statistically enriched under the true evaluator and updates remain conservative. On NAVSIM, CLOVER achieves 94.5 PDMS and 90.4 EPDMS, establishing a new state-of-the-art performance. On the more challenging NavHard split, it obtains 48.3 EPDMS, matching the strongest reported result. On supplementary nuScenes open-loop evaluation, CLOVER achieves the lowest L2 error and collision rate among compared methods. Code and generated data will be released at <https://github.com/WilliamXuanYu/CLOVER>.

1 Introduction

End-to-end autonomous driving planning has become a promising paradigm for mapping sensor observations directly to future ego trajectories [15, 18, 7, 12, 27, 23, 4]. However, planning is not a standard trajectory-regression problem. Most planners are trained to imitate a single logged human trajectory, whereas they are evaluated by rule-based planning metrics such as NAVSIM PDMS/EPDMS [9, 2], which measure safety, feasibility, progress, and comfort. A trajectory close to the logged path may still collide, leave the drivable area, or induce unsafe interactions, while another

*Corresponding author.

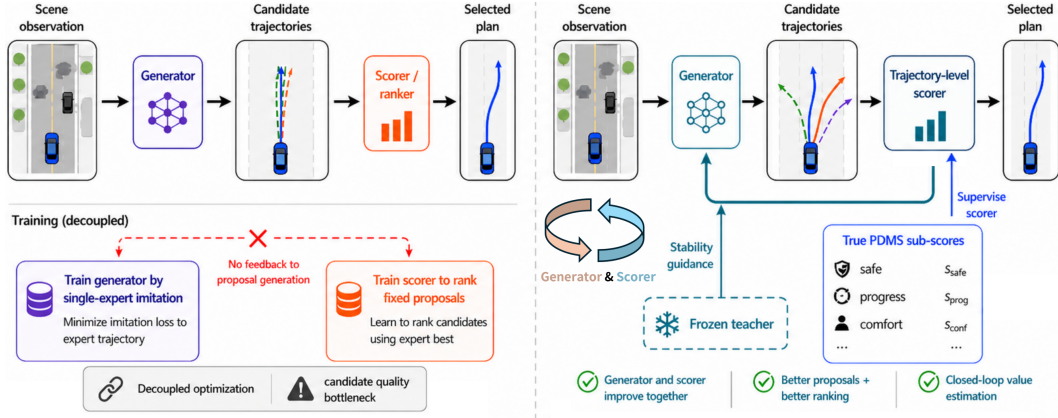


Figure 1: **CLOVER closes the loop between proposal generation and trajectory ranking. Left:** Conventional proposal-selection planners generate candidate trajectories and select the final plan with a learned scorer or ranker. During training, however, the generator is typically optimized by single-expert imitation, so ranking feedback does not explicitly reshape the proposal distribution toward higher-value regions. **Right:** CLOVER trains a trajectory-level scorer with true planning-metric sub-scores and conservatively refines the generator through teacher-guided self-distillation. This closed-loop design improves candidate-set coverage and ranking quality together, increasing the chance that high-value trajectories are both generated and selected.

trajectory farther from the demonstration may remain valid and high-scoring. This training–evaluation mismatch limits purely imitation-based planning under closed-loop-style metrics.

The mismatch is particularly important for proposal-selection planners. These methods first generate multiple candidate trajectories and then select one using a learned scorer or ranker. Their final performance is constrained by two coupled factors: whether the generator covers high-quality alternatives, and whether the scorer can identify them. Single-trajectory supervision can collapse candidates around one realized behavior, even when multiple decisions are valid in the same scene, such as maintaining speed, braking, or choosing a different interaction strategy. Conversely, even a strong candidate set can fail if the scorer ranks unsafe or infeasible trajectories above better alternatives. Thus, effective proposal-selection planning requires both broad proposal coverage and reliable use of scorer feedback.

We propose **CLOVER**, a **C**losed-**L**oop **V**alue **E**stimation and **R**anking framework for proposal-selection based end-to-end autonomous driving planning. Given multi-view camera observations and ego state, CLOVER uses a lightweight generator–scorer architecture: the generator produces candidate future ego trajectories, and a trajectory-level scorer predicts planning-metric sub-scores for each candidate. At inference time, candidates are ranked by the composed predicted score and the top-ranked trajectory is selected. During training, the true rule-based evaluator supervises scorer learning and generator refinement, but the evaluator is not required at deployment.

CLOVER addresses the training–evaluation mismatch in two stages. First, it expands single-trajectory imitation into **set-level proposal coverage**. Evaluator-filtered pseudo-expert trajectories are constructed from diverse trajectory families, including different lateral offsets, speed profiles, and boundary behaviors, and the generator is trained to cover them while retaining the logged trajectory as a driving prior. This exposes the generator to multiple feasible driving modes and raises the oracle upper bound of the proposal set. Second, CLOVER performs **conservative closed-loop self-distillation**. The scorer is fitted to true evaluator sub-scores on generated proposals; a frozen teacher then provides top- k and vector-Pareto proposal targets, which the student generator covers with stability regularization. This lets the scorer guide generator improvement without unrestricted maximization of an imperfect learned reward.

A key question is when such scorer-mediated refinement is trustworthy. CLOVER does not require a globally perfect scorer. Our analysis shows that it is sufficient for scorer-selected targets to be statistically better under the true evaluator than the current proposal distribution. Under this selected-set enrichment condition, conservative set-level distillation increases the generator’s probability mass

on high-quality trajectories. This provides a direct condition under which an imperfect learned scorer can serve as a useful mediator for proposal refinement. We further validate this condition empirically, showing that scorer-selected targets are enriched under the true evaluator and that Stage-2 refinement improves proposal quality without collapsing diversity.

We evaluate CLOVER on NAVSIM v1, NAVSIM v2 EPDMS, NavHard, and supplementary nuScenes open-loop evaluation. CLOVER matches or surpasses the strongest reported results across the main NAVSIM-style benchmarks, achieving 94.5 PDMS and 90.4 EPDMS on NAVSIM and 48.3 EPDMS on NavHard, while also obtaining strong nuScenes open-loop results. Further analyses show that CLOVER improves both proposal quality and proposal diversity, indicating that its gains come from a stronger candidate distribution rather than only from selecting a better top-1 trajectory. Although CLOVER superficially resembles actor-critic methods, it does not interact with the environment or perform trial-and-error exploration; all feedback is computed offline from logged scenes and a rule-based evaluator.

Our contributions are summarized as follows:

- We propose **CLOVER**, a closed-loop value estimation and ranking framework that couples proposal generation and trajectory scoring.
- We introduce evaluator-filtered pseudo-expert coverage supervision, extending single-trajectory imitation to set-level multi-modal proposal training and improving candidate-set quality.
- We develop conservative closed-loop self-distillation, alternating between scorer fitting on true planning sub-scores and teacher-guided generator refinement with top- k , vector-Pareto, and stability objectives.
- We analyze when imperfect-scorer guidance is reliable under selected-set enrichment, and empirically show that CLOVER improves proposal quality while preserving diversity across NAVSIM-style evaluations.

2 Related Work

2.1 End-to-end Autonomous Driving Planning

End-to-end autonomous driving planning aims to predict future ego trajectories directly from sensor observations. Early methods typically mapped front-view images to low-level control commands, while recent approaches use multi-view encoders, Transformer decoders, BEV representations, trajectory queries, and vision pretraining to unify scene understanding and planning [15, 18, 7, 12, 27, 23, 4]. These methods have significantly improved planning performance, especially under modern evaluation protocols such as NAVSIM [9, 2]. However, most end-to-end planners are still trained primarily by imitating logged expert trajectories. Such supervision is stable and data-efficient, but it is not fully aligned with rule-based planning metrics that evaluate safety, feasibility, progress, and comfort. CLOVER differs by explicitly using evaluator-provided planning feedback to train a trajectory-level value estimator and to refine the candidate proposal distribution, rather than relying only on single-trajectory imitation.

2.2 Multimodal Trajectory Generation and Proposal-selection Planning

Driving behavior is inherently multi-modal: in the same scene, multiple future trajectories, such as maintaining speed, yielding, braking, or choosing a slightly different interaction strategy, may all be valid. Proposal-selection planners address this by generating a set of candidate trajectories and selecting one for execution. One line of work uses discretized trajectory vocabularies or anchor sets, such as Hydra-MDP [24] and VADv2 [4], which provide a structured candidate space but are constrained by the coverage of the predefined vocabulary. Another line uses generative models, such as DiffusionDrive [27] and GoalFlow [37], to produce diverse trajectories with greater flexibility, often at higher computational cost. Proposal-centric planners such as iPad [12] refine candidate trajectories online and offer an efficient alternative. Despite these advances, proposal generation is often still supervised mainly by the single logged trajectory, which can limit candidate-set coverage when multiple high-quality behaviors are possible. CLOVER addresses this limitation by construct-

ing evaluator-filtered pseudo-expert trajectories and training the generator with set-level coverage supervision, thereby improving both proposal diversity and candidate-set quality.

2.3 Learned Trajectory Scoring

A key component of proposal-selection planning is trajectory scoring: given a candidate set, the planner must identify the safest and most effective trajectory. Learned cost functions, ranking modules, and trajectory scorers have been used to improve selection quality, especially when paired with rich proposal sets [38, 20, 25]. However, trajectory scoring in autonomous driving is challenging because the true planning score depends on map constraints, future obstacle states, interaction rules, and non-differentiable metric computations. A learned scorer must estimate this quality from the current observation and candidate trajectory, and directly maximizing such an imperfect score can exploit scorer errors or reduce diversity. CLOVER treats the scorer as a closed-loop value estimator trained on evaluator-provided sub-scores, and uses it conservatively: instead of unrestricted score maximization, the generator is refined through teacher-selected top- k and vector-Pareto proposal targets with stability regularization.

2.4 Relation to Reinforcement Learning

CLOVER shares a superficial similarity with actor-critic reinforcement learning: a generator proposes trajectories and a scorer evaluates them. However, CLOVER is not a reinforcement learning method. Classic reinforcement learning relies on environment interaction, trial-and-error exploration, and reward optimization over visited states. In contrast, CLOVER operates offline on logged driving scenes; all feedback is computed from generated candidate trajectories using a rule-based planning evaluator, and no online environment interaction or exploration is performed. Recent works such as Evadrive [19] explore RL-style planning for autonomous driving, whereas CLOVER focuses on closed-loop value estimation and conservative self-distillation for proposal-selection planning. Thus, CLOVER is better viewed as an offline evaluator-guided ranking and proposal-refinement framework rather than an RL-based planner.

3 Method

Overview. Figure 2 summarizes CLOVER. CLOVER follows a proposal-selection formulation: a generator produces a set of candidate ego trajectories, and a trajectory-level scorer ranks them by predicted planning-metric sub-scores. Training has two stages. Stage 1 expands proposal support with evaluator-filtered pseudo-expert coverage supervision and scorer pretraining. Stage 2 performs conservative closed-loop self-distillation by alternating scorer fitting on evaluator-provided sub-scores and teacher-guided generator refinement.

3.1 Proposal Selection and Scoring

Given a driving observation o , including multi-view camera images and ego-state information, CLOVER predicts future ego trajectories in the ego-centric frame. Each trajectory is represented as $\tau = \{(x_t, y_t, \theta_t)\}_{t=1}^T$, where (x_t, y_t, θ_t) denotes the ego pose at future step t . In our implementation, the prediction horizon is 4 seconds with a 0.5-second interval, resulting in $T = 8$ future poses.

Instead of outputting a single trajectory, the generator produces K candidates

$$\mathcal{T}_\theta(o) = G_\theta(o) = \{\tau_i\}_{i=1}^K. \quad (1)$$

A trajectory-level scorer predicts planning-metric sub-scores for each candidate,

$$\hat{\mathbf{r}}_i = S_\phi(o, \tau_i), \quad (2)$$

covering components such as collision avoidance, drivable-area compliance, time-to-collision, progress, and comfort. These predicted sub-scores are composed by the official PDMS rule $\Gamma_{\text{PDMS}}(\cdot)$, and inference selects

$$\tau^* = \arg \max_{\tau_i \in \mathcal{T}_\theta(o)} \Gamma_{\text{PDMS}}(S_\phi(o, \tau_i)). \quad (3)$$

During training and analysis, the rule-based evaluator E assigns true targets $\mathbf{r}_i^{\text{true}} = E(o, \tau_i)$. Since E involves map queries, future-obstacle states, geometric checks, and discrete rules, it is

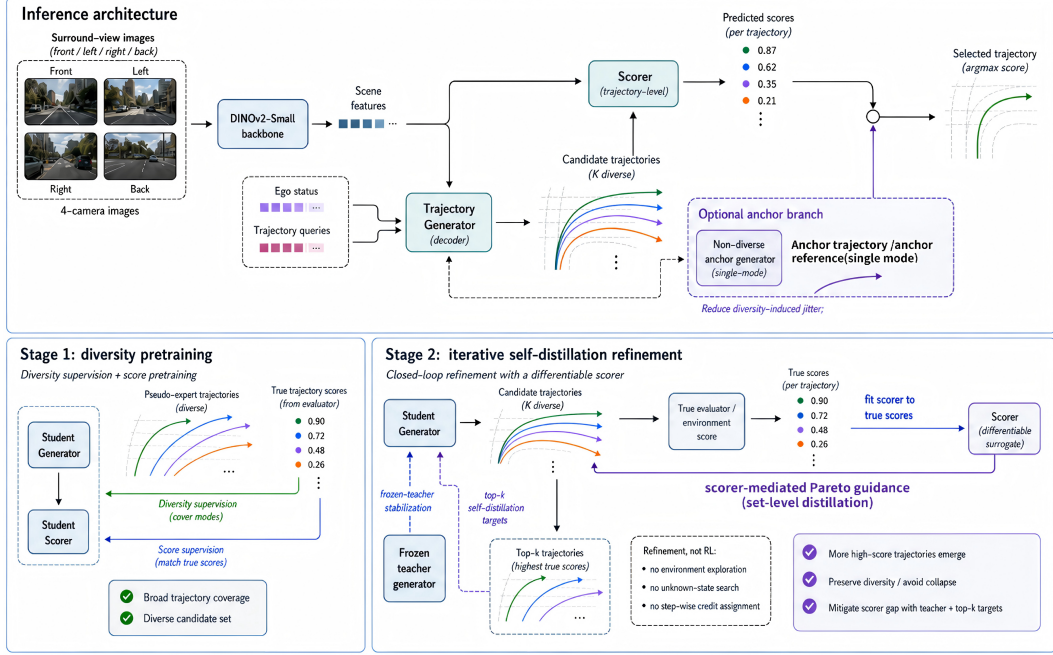


Figure 2: **Overview of CLOVER.** At inference time, multi-view images and ego state are encoded into scene features. A generator produces K candidate trajectories, and a trajectory-level scorer ranks them by predicted planning-metric sub-scores. Stage 1 expands proposal coverage with evaluator-filtered pseudo-expert trajectories. Stage 2 fits the scorer to evaluator-provided sub-scores and refines the generator through teacher-guided top- k and vector-Pareto distillation with stability regularization. An optional anchor-assisted soft reranking module can be used for EPDMS evaluation to reduce diversity-induced jitter among near-tied candidates.

non-differentiable with respect to the generator. CLOVER uses the evaluator only to supervise the scorer and construct distillation targets during training; it is not required at deployment. Exact PDMS/EPDMS definitions are provided in Appendix B.

We instantiate this formulation with a lightweight DrivoR-style generator–scorer architecture [20]. A DINOv2-Small visual encoder with LoRA fine-tuning maps 4-camera images to compact scene tokens, which are combined with ego-state features and decoded by a Transformer trajectory generator with learnable trajectory queries. The scorer embeds each candidate trajectory through its explicit waypoints, attends to the same scene tokens, and predicts planning-metric sub-scores. Thus, the generator determines which behaviors are available for selection, while the scorer estimates their closed-loop trajectory quality. We use $K = 64$ candidates unless otherwise specified; implementation details are in Appendix E.

3.2 Stage 1: Evaluator-filtered Proposal Coverage

Single-trajectory imitation treats the logged future trajectory as the only positive target, which is insufficient for training a high-coverage proposal generator. CLOVER therefore constructs a training-time pseudo-expert set $\mathcal{E}(o)$ for each scene. These pseudo-experts are not simple perturbations of the logged trajectory. Instead, they are generated from interpretable candidate families using privileged metric-cache information available during training, including route centerlines, drivable-area maps, and future obstacle occupancy. The candidate pool covers diverse lateral offsets, acceleration and deceleration profiles, stop-go behaviors, approach-brake trajectories, boundary cases, and overshoot-style motions. Candidates are pre-filtered by lightweight drivable-area and occupancy checks, scored by the evaluator, and selected with a coverage-aware strategy over score patterns and bins.

During training, we further keep high-quality candidates above a score threshold and apply greedy farthest-point sampling in trajectory space to obtain at most M pseudo-experts, $\mathcal{E}(o) = \{e_j\}_{j=1}^M$.

This set is used only during training and is never required at inference time. Detailed generation rules and qualitative examples are provided in Appendix A.

Given proposals $\mathcal{T}_\theta(o) = \{\tau_i\}_{i=1}^K$ and the logged trajectory τ^{gt} , Stage 1 optimizes

$$\begin{aligned} \mathcal{L}_{\text{stage1}} = & \underbrace{\lambda_{\text{gt}} \min_i \|\tau_i - \tau^{\text{gt}}\|_1}_{\mathcal{L}_{\text{gt}}} + \underbrace{\lambda_{\text{pe}} \frac{1}{|\mathcal{E}(o)|} \sum_{e \in \mathcal{E}(o)} \min_i \|\tau_i - e\|_1}_{\mathcal{L}_{\text{pe}}} \\ & + \underbrace{\lambda_{\text{score}} \frac{1}{|\mathcal{Q}(o)|} \sum_{\tau \in \mathcal{Q}(o)} \ell(S_\phi(o, \tau), E(o, \tau))}_{\mathcal{L}_{\text{score}}}. \end{aligned} \quad (4)$$

Here, $\mathcal{L}_{\text{gt}}$ preserves the logged driving prior, $\mathcal{L}_{\text{pe}}$ encourages the proposal set to cover multiple pseudo-expert modes, and $\mathcal{L}_{\text{score}}$ pretrains the scorer to predict evaluator-provided sub-scores. The loss ℓ uses component-specific classification or regression targets. This stage expands the candidate distribution beyond a single realized human trajectory while retaining stable imitation supervision.

3.3 Stage 2: Conservative Closed-loop Self-distillation

Stage 2 refines the expanded proposal distribution using evaluator feedback. Directly maximizing a learned scalar score can exploit scorer errors or collapse diversity, so CLOVER uses an alternating procedure initialized from Stage 1: a scorer phase fits the value estimator on current proposals, and a generator phase distills high-value teacher proposals under stability regularization.

Scorer phase. The current generator produces $\mathcal{T}_\theta(o) = \{\tau_i\}_{i=1}^K$, and the evaluator provides true sub-score targets $\mathbf{r}_i^{\text{true}} = E(o, \tau_i)$. The scorer minimizes

$$\mathcal{L}_{\text{critic}} = \frac{1}{K} \sum_{i=1}^K \ell(S_\phi(o, \tau_i), \mathbf{r}_i^{\text{true}}), \quad (5)$$

distilling the non-differentiable evaluator into a differentiable trajectory-level value estimator on the current proposal distribution.

Generator phase. A frozen teacher generates $\mathcal{T}^{\text{tea}}(o) = \{\tau_i^{\text{tea}}\}_{i=1}^K$. The scorer evaluates these proposals and constructs two target sets. The first is the top- k set $\mathcal{K}^{\text{tea}}$, selected by the composed predicted score $\Gamma_{\text{PDMS}}(S_\phi(o, \tau))$. The second is the vector-Pareto set $\mathcal{P}^{\text{tea}}$, consisting of non-dominated proposals in predicted sub-score space. A trajectory τ_a dominates τ_b if $r_a^m \geq r_b^m$ for all components m and $r_a^m > r_b^m$ for at least one component.

The student proposal set $\mathcal{T}^{\text{stu}}(o) = \{\tau_i^{\text{stu}}\}_{i=1}^K$ is trained to cover these targets with

$$\mathcal{L}_{\text{set}}(\mathcal{T}^{\text{stu}}, \mathcal{A}) = \frac{1}{|\mathcal{A}|} \sum_{\tau \in \mathcal{A}} \min_i \|\tau_i^{\text{stu}} - \tau\|_1, \quad \mathcal{A} \in \{\mathcal{K}^{\text{tea}}, \mathcal{P}^{\text{tea}}\}. \quad (6)$$

The generator objective is

$$\begin{aligned} \mathcal{L}_{\text{gen}} = & \lambda_{\text{traj}} \mathcal{L}_{\text{gt}} + \lambda_{\text{topk}} \mathcal{L}_{\text{set}}(\mathcal{T}^{\text{stu}}, \mathcal{K}^{\text{tea}}) + \lambda_{\text{pareto}} \mathcal{L}_{\text{set}}(\mathcal{T}^{\text{stu}}, \mathcal{P}^{\text{tea}}) \\ & + \lambda_{\text{stab}} \frac{1}{K} \sum_{i=1}^K \|\tau_i^{\text{stu}} - \text{sg}(\tau_i^{\text{tea}})\|_1. \end{aligned} \quad (7)$$

The top- k term concentrates probability mass on high-scoring proposals, the vector-Pareto term preserves safety-progress-comfort trade-offs, and the stability term keeps refinement conservative by discouraging drift away from the teacher proposal distribution.

3.4 Inference and Optional Anchor Reranking

At inference time, CLOVER uses only the generator and learned scorer: candidates are generated, scored, and ranked by the composed predicted metric score. The evaluator, pseudo-expert generator, and self-distillation procedure are used only during training and analysis.

For NAVSIM v2 EPDMS, we optionally apply anchor-assisted soft reranking to reduce diversity-induced jitter among near-tied high-quality candidates. This is useful because EPDMS includes

extended-comfort terms that depend on cross-frame decision consistency, whereas the scorer predicts per-trajectory planning-metric sub-scores. The anchor is produced by a single-mode planner trained with the same architecture and data protocol but with diversity-related settings disabled and only standard single-trajectory imitation retained. It uses no future information or evaluator scores at inference time; it only provides a stable reference trajectory. Candidates are reranked by combining the learned scorer score with a continuous penalty for position and heading deviation from the anchor. This soft reranking does not discard candidates with hard thresholds; it only biases selection toward temporally stable choices among near-tied high-quality proposals. The exact formula and parameter sweep are provided in Appendix C.

4 Theoretical Analysis

We provide a concise theoretical justification for when CLOVER’s scorer-mediated refinement can improve the generator. The key question is not whether the learned scorer is globally perfect, but whether the targets selected by the scorer are statistically better under the true evaluator than the current proposal distribution. We therefore focus on a selected-set enrichment condition: if the scorer-selected top- k or vector-Pareto targets contain more true high-quality trajectories than the current proposal set, then conservative set-level distillation increases the generator’s support on high-quality trajectories. Full proofs, auxiliary results on approximate monotonicity, scorer refitting, and Pareto consistency, and additional empirical validation are provided in Appendix D.

All scores are normalized to $[0, 1]$. For a scene o , let

$$\mu_t^o = \frac{1}{K} \sum_{i=1}^K \delta_{\tau_i^t(o)}$$

denote the empirical distribution of the current K proposals. Let $A_t(o)$ be the scorer-selected target set, such as the top- k or vector-Pareto teacher proposals, and let ν_t^o be the empirical distribution over $A_t(o)$. For a high-score region

$$\mathcal{H}_o = \{\tau : R^*(o, \tau) \geq r_{\text{high}}\},$$

define $p_t(o) = \mu_t^o(\mathcal{H}_o)$ and $q_t(o) = \nu_t^o(\mathcal{H}_o)$.

Theorem 1 (Selected-set enrichment improves high-score support) *Assume that the scorer-selected target set is enriched with true high-score trajectories:*

$$q_t(o) \geq p_t(o) + \xi_t(o), \quad \xi_t(o) > 0.$$

Assume that the generator update is conservative, i.e.,

$$\text{TV}(\mu_{t+1}^o, (1 - \alpha_t)\mu_t^o + \alpha_t\nu_t^o) \leq \eta_t(o), \quad \alpha_t \in [0, 1].$$

Then

$$p_{t+1}(o) \geq p_t(o) + \alpha_t\xi_t(o) - \eta_t(o).$$

Therefore, if $\alpha_t\xi_t(o) > \eta_t(o)$, Stage-2 refinement increases the probability mass assigned to high-score trajectories.

Similarly, if the selected targets have higher true expected score,

$$\mathbb{E}_{\nu_t^o}[R^*] - \mathbb{E}_{\mu_t^o}[R^*] \geq \beta_t(o) > 0,$$

then

$$\mathbb{E}_{\mu_{t+1}^o}[R^*] \geq \mathbb{E}_{\mu_t^o}[R^*] + \alpha_t\beta_t(o) - \eta_t(o).$$

Implication for CLOVER. The theorem states a weak and testable condition: the scorer does not need to rank every trajectory correctly, nor does it need to be accurate over the entire trajectory space. It only needs to select target sets that are statistically enriched with true high-quality trajectories under the evaluator. This matches CLOVER’s design. Stage 1 provides broad proposal support, the scorer phase calibrates the value estimator on generated proposals, and the generator phase conservatively distills scorer-selected top- k and vector-Pareto targets rather than directly maximizing a learned scalar reward. The stability term controls the deviation from the teacher proposal distribution, corresponding to the conservative-update error $\eta_t(o)$.

Empirical support. We empirically validate the two premises behind Theorem 1: the proposal pool must already contain high-quality modes, and the scorer-selected targets must be enriched under the true evaluator. On 12,146 NAVSIM scenes with 64 proposals per scene, the pooled proposal distribution contains substantial high-score support, with 35.42% full-score proposals. More importantly, high-confidence scorer-selected proposals are much more likely to be truly high scoring: among proposals with predicted score $s \geq 0.95$, the true mean score is 0.9753, $P(R^* \geq 0.90) = 96.69\%$, and $P(R^* = 1) = 69.74\%$. Thus, for the full-score region $\mathcal{H}_o = \{\tau : R^*(o, \tau) = 1\}$, the empirical enrichment gap is approximately $69.74\% - 35.42\% = 34.32\%$, directly matching the condition $q_t(o) \geq p_t(o) + \xi_t(o)$.

We further evaluate whether the scorer is useful for target selection, rather than globally perfect trajectory scoring. For fixed groups with true score ≥ 0.95 versus ≤ 0.50 , the scorer reaches 94.72% pairwise ranking accuracy, and the elite/reject margin condition $\gamma_o > 2\epsilon_{p90,o}$ holds in 82.37% of scenes. Finally, when the scorer is used to select target sets from the proposal pool, the scorer-selected Top-8 and Top-16 sets achieve mean best true scores of 0.9656 and 0.9730, respectively, supporting the expected-score version of Theorem 1. These diagnostics are not used as final benchmark results; rather, they verify that the target-selection step provides evaluator-enriched supervision for Stage-2 distillation. Overall, Stage 2 does not rely on a globally perfect scorer or on discovering high-quality modes from scratch. Instead, it reallocates generator probability mass toward evaluator-enriched targets selected from an already broad proposal space. Additional empirical validation is provided in Appendix D.

5 Experiments

5.1 Experimental Setup

We evaluate CLOVER on NAVSIM v1, NAVSIM v2 EPDMS, NavHard, and supplementary nuScenes open-loop planning. Our main evaluation follows NAVSIM-style planning metrics, where trajectory quality is measured by rule-based safety, progress, and comfort components rather than only displacement error to the logged trajectory. NAVSIM v1 reports PDMS, while NAVSIM v2 reports EPDMS on the navtest split [9, 2]. We further evaluate on the more challenging navhard-two-stage split using EPDMS. For supplementary open-loop validation on nuScenes [1], we follow the ST-P3 and UniAD protocols and report L2 displacement error and collision rate [14, 16]. Exact PDMS/EPDMS definitions are provided in Appendix B.

Unless otherwise specified, all experiments use the same DrivoR-style generator–scorer architecture [20]. The model uses 4-camera images without LiDAR, a DINOv2 ViT-S visual encoder with LoRA fine-tuning [28, 13], 64 trajectory proposals, and a 4-second horizon sampled every 0.5 seconds. Stage 1 is trained with pseudo-expert coverage and scorer pretraining, and Stage 2 is initialized from Stage 1 and trained with alternating scorer-fitting and generator-refinement phases. Because CLOVER modifies training but keeps the deployed generator–scorer inference graph unchanged, it introduces no additional inference-time modules over the base DrivoR-style architecture. With $K = 64$ proposals, the deployed model runs at about 110 ms per scene on a single NVIDIA A100 GPU in the NAVSIM-v2 setting. Detailed architecture, hyperparameters, and training cost are provided in Appendix E.

5.2 Main Results

Tables 1–4 summarize results on NAVSIM v1, NAVSIM v2 navtest, NavHard, and supplementary nuScenes open-loop evaluation. CLOVER achieves **94.5 PDMS** on NAVSIM v1, improving over the strongest generator–scorer baseline DrivoR [20] by 0.8 points and ranking first or second across all PDMS sub-scores. Under NAVSIM v2, it obtains **90.4 EPDMS** with the updated official implementation and **87.2 EPDMS*** with the original evaluation code. On the challenging navhard-two-stage split, CLOVER reaches **48.3 EPDMS**, matching the strongest reported result. It also achieves strong supplementary nuScenes open-loop results, including the lowest ST-P3 L2 error and best or tied-best collision rate.

We also repeat the main CLOVER training with three random seeds and observe negligible variation: all runs differ by less than 0.02 PDMS and round to the same one-decimal benchmark score; detailed seed-stability results are provided in Appendix G.

Table 1: **NAVSIM v1 benchmark.** Comparison with published and recent methods under the official PDMS metric. Higher is better for all columns. CLOVER achieves the best PDMS and ranks first or second across all sub-scores, nearly matching the human-driver reference.

Method		NC	DAC	TTC	Comf.	EP	PDMS
PDMS-Closed	[8] PMLR'23	94.6	99.8	89.9	86.9	99.9	89.1
Human driver	[9] NeurIPS'24	100	100	100	99.9	87.5	94.8
Ego-stat. MLP	[9] NeurIPS'24	93.0	77.3	83.6	100	62.8	65.6
UniVLA	[33] arXiv'25	96.9	91.1	91.7	96.7	76.8	81.7
DrivingGPT	[5] ICCV'24	98.9	90.7	94.9	95.6	79.7	82.4
UniAD	[15] CVPR'23	97.8	91.9	92.9	100	78.8	83.4
LTF	[7] TPAMI'22	97.4	92.8	92.4	100	79.0	83.8
PARA-Drive	[34] CVPR'24	97.9	92.4	93.0	99.8	79.3	84.0
DriveX-S	[29] ICCV'25	97.5	94.0	93.0	100	79.7	84.5
World4Drive	[40] ICCV'25	97.4	94.3	92.8	100	79.9	85.1
DRAMA	[39] ISRR'24	98.0	93.1	94.8	100	80.1	85.5
VAD-v2	[4] arXiv'24	98.1	94.8	94.3	100	80.6	86.2
PRIX	[35] arXiv'25	98.1	96.3	94.1	100	82.3	87.8
DiffusionDrive	[27] CVPR'25	98.2	96.2	94.7	100	82.2	88.1
DIVER	[31] arXiv'25	98.5	96.5	94.9	100	82.6	88.3
AutoVLA	[42] NeurIPS'25	98.4	95.6	98.0	99.9	81.9	89.1
DriveVLA-W0	[22] ICLR 26'	98.7	99.1	95.3	99.3	83.3	90.2
ReCogDrive	[23] ICLR 26'	97.9	97.3	94.9	100	87.3	90.8
Hydra-MDP++	[21] arXiv'25	98.6	98.6	95.1	100	85.7	91.0
iPad	[12] arXiv'25	98.6	98.3	94.9	100	88.0	91.7
Centaur	[30] arXiv'25	99.5	98.9	98.0	100	85.9	92.6
DriveSuprim	[38] arXiv'25	98.6	98.6	95.5	100	91.3	93.5
DrivoR	[20] CVPR'26	99.0	98.9	96.7	100	90.0	93.7
Clover(ours)		99.1	99.0	96.9	100	91.7	94.5

Table 2: **NAVSIM v2 EPDMS on navtest.** Comparison under the Extended PDMS metric. EPDMS* denotes results computed with the original NAVSIM v2 evaluation code before applying the human-behavior filtering fix, while EPDMS denotes results computed with the updated official implementation. Higher is better for all columns.

Method	NC ↑	DAC ↑	DDC ↑	TL ↑	EP ↑	TTC ↑	LK ↑	HC ↑	EC ↑	EPDMS* ↑	EPDMS ↑
Ego-MLP [26]	93.1	77.9	92.7	99.6	86.0	91.5	89.4	98.3	85.4	64.0	–
Transfuser [7]	96.9	89.9	97.8	99.7	87.1	95.4	92.7	98.3	87.2	76.7	–
ARTEMIS [11]	98.3	95.1	98.6	99.8	81.5	97.4	96.5	98.3	98.3	83.1	–
DiffusionDriveV2 [43]	97.7	96.6	99.2	99.8	88.9	97.2	96.0	97.8	91.0	85.5	87.5
Hydra-MDP++ [21]	98.4	98.0	99.4	99.8	87.5	97.7	95.3	98.3	77.4	85.1	–
DriveSuprim [38]	97.8	97.9	99.5	99.9	90.6	97.1	96.6	98.3	77.9	86.0	–
SparseDriveV2 [32]	98.1	98.1	99.6	99.8	91.1	97.3	96.9	98.2	78.4	86.7	90.1
Clover(ours)	99.4	99.3	99.5	99.8	86.9	98.9	95.2	98.2	75.5	87.2	90.4

5.3 Proposal Quality and Diversity Analysis

A central goal of CLOVER is to improve the proposal set itself, rather than only rerank a fixed set of candidates. We therefore analyze the 64 generated proposals across three stages: the DrivoR baseline, Stage 1 pseudo-expert diversity pretraining, and Stage 2 self-distillation refinement. All statistics are computed on 12,146 common scenes, using true PDMS scores from the evaluator for analysis only. Following the proposal-diversity protocol of Curious-VLA [3], we report pairwise ADE/FDE and additional proposal-set quality metrics in Table 5.

Table 5 shows a clear stage-wise behavior. Stage 1 greatly expands the proposal space: compared with the baseline, it increases Oracle@64 from 0.9933 to 0.9976, Pairwise ADE from 1.80 to 5.97, Pairwise FDE from 4.47 to 10.70, and Endpoint Area from 5.78 to 46.18. This confirms that pseudo-expert coverage supervision raises the oracle upper bound, although it also introduces a heavier low-score tail.

Stage 2 then refines this expanded distribution. It achieves the best scorer-selected PDMS, improving from 0.9369 for the baseline and 0.9413 for Stage 1 to 0.9448. It also obtains the highest mean

Table 3: **NAVSIM v2 navhard-two-stage.** Comparison on the challenging NavHard split using EPDMS. Stage-1 and Stage-2 columns report sub-scores from the two-stage evaluation procedure, and EPDMS denotes the final aggregated score. Higher is better for all columns.

Method	Stage 1									Stage 2									EPDMS
	NC	DAC	DDC	TLC	EP	TTC	LK	HC	EC	NC	DAC	DDC	TLC	EP	TTC	LK	HC	EC	
RAP-DINO [10]	97.1	94.4	98.8	99.8	83.9	96.9	94.7	96.4	66.2	83.2	83.9	87.4	98.0	86.9	80.4	52.3	95.2	52.4	39.6
GTRS-D [25]	98.9	96.2	99.4	99.3	72.9	98.9	95.1	96.9	39.1	91.2	89.4	94.4	98.8	69.5	90.0	54.3	94.0	48.7	45.0
GTRS-A [25]	98.9	95.1	99.1	99.6	76.2	99.1	94.9	97.6	54.2	88.1	88.8	89.3	98.9	98.9	85.9	53.7	96.8	56.9	45.4
DrivR [20]	98.8	95.1	98.9	100	72.6	98.7	94.0	97.6	73.3	90.2	88.4	91.9	98.6	70.0	88.0	50.1	98.5	76.2	48.3
Clover (ours)	99.4	98.0	99.9	100	75.7	98.9	94.4	97.1	56.0	89.8	88.2	93.2	99.0	74.2	87.7	52.6	97.0	57.3	48.3

Table 4: **Open-loop planning evaluation on nuScenes.** We report L2 displacement error and collision rate under the ST-P3 and UniAD protocols. Lower is better for all metrics. This benchmark is used as supplementary open-loop validation.

Method	ST-P3 metrics		UniAD metrics	
	L2 (m) ↓	Collision (%) ↓	L2 (m) ↓	Collision (%) ↓
ST-P3 [14]	2.11	0.71	–	–
VAD [18]	0.37	0.14	–	–
UniAD [16]	0.69	0.12	1.03	0.31
EMMA [17]	0.32	–	–	–
OpenEMMA [36]	2.81	–	–	–
AutoVLA [42]	0.48	0.13	0.86	0.35
Impromptu VLA [6]	0.33	0.13	0.67	0.38
OpenDriveVLA [41]	0.33	0.10	0.67	0.30
Clover (ours)	0.31	0.10	0.65	0.30

proposal score, 0.8277, and reduces the number of low-score proposals with PDMS < 0.50 to 6.83. At the same time, it preserves substantially more diversity than the baseline, with Pairwise ADE 5.20 versus 1.80 and qualified cluster count 8.71 versus 6.02. These results support the intended division of labor: Stage 1 expands proposal support, while Stage 2 converts the expanded space into a higher-quality and better-ranked proposal distribution. Additional qualitative comparisons and scorer-only diagnostic experiments are provided in Appendices H.1 and F.

5.4 Ablation Studies

We ablate the main components of CLOVER on NAVSIM v1 PDMS and NAVSIM v2 EPDMS. The ablations examine four design choices: pseudo-expert proposal coverage, alternating closed-loop refinement, vector-Pareto teacher-set construction, and anchor-assisted reranking.

Main components. Table 6 shows that the two stages are complementary. The single-expert baseline obtains 93.7 PDMS. Adding pseudo-expert diversity supervision improves the score to 94.1, while applying closed-loop refinement without the diversity stage yields only 93.8. The full model reaches 94.5, suggesting that scorer-guided refinement is most effective when Stage 1 has already expanded the proposal support.

Stage-2 refinement design. Table 7 studies the training strategy for closed-loop refinement. A non-iterative generator/scorer update collapses severely, indicating that directly updating the generator against a changing imperfect scorer is unstable. Self-distillation alone remains stable but saturates at 93.8–94.0 PDMS, depending on the number of distilled trajectories. The full alternating procedure achieves 94.5, supporting the need to repeatedly refit the scorer to evaluator-provided sub-scores before using it to guide generator refinement.

Teacher-set construction. Table 8 compares different target sets for Stage-2 generator refinement. Using scalar real-PDMS top- k targets gives 93.9 PDMS and relatively low proposal diversity, suggesting that scalar top- k supervision can over-concentrate the student around a narrow set of high-score modes. Adding distance suppression improves both PDMS and diversity, but still underperforms vector-Pareto guidance. Our vector-Pareto targets achieve the best PDMS and the highest high-quality

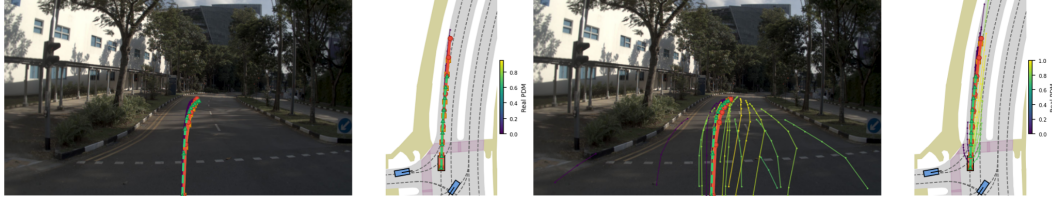


Figure 3: **Qualitative proposal diversity.** Compared with the DrivoR baseline, which concentrates candidates around a narrow mode, CLOVER covers a wider set of feasible trajectory branches. Many of these diverse candidates remain high-scoring under the evaluator, showing that the additional diversity is not merely caused by low-quality outliers.

Table 5: **Stage-wise proposal quality and diversity analysis.** Metrics are computed over 64 generated proposals on 12,146 common scenes. Stage 1 greatly expands the proposal space and achieves the highest oracle upper bound, while Stage 2 refines the expanded distribution, obtaining the best selected score and proposal-set quality while preserving much higher diversity than the baseline. Higher is better except for Gap and Count(PDMS < 0.50).

Metric	Baseline	Stage 1	Stage 2
Selected PDMS $\uparrow$	0.9369	0.9413	0.9448
Oracle@64 PDMS $\uparrow$	0.9933	0.9976	0.9939
Gap(Oracle - Selected) $\downarrow$	0.0564	0.0563	0.0495
Mean PDMS over 64 $\uparrow$	0.7972	0.7374	0.8277
Std PDMS over 64	0.2152	0.3013	0.2235
Count(PDMS > 0.95) $\uparrow$	30.94	24.78	31.97
Count(PDMS < 0.50) $\downarrow$	9.05	12.21	6.83
Pairwise ADE $\uparrow$	1.80	5.97	5.20
Pairwise FDE $\uparrow$	4.47	10.70	8.25
Endpoint Area $\uparrow$	5.78	46.18	25.56
Trajectory Effective Rank $\uparrow$	1.14	1.24	1.27
Qualified Cluster Count@2m $\uparrow$	6.02	10.91	8.71
Top-6 Real Pairwise ADE $\uparrow$	0.91	2.11	1.75
Top-6 Real Pairwise FDE $\uparrow$	2.20	3.55	2.59

proposal diversity, indicating that preserving multi-objective trade-offs among safety, progress, and comfort is beneficial for proposal refinement.

Anchor-assisted reranking. Table 9 evaluates the optional anchor-assisted soft reranking used for NAVSIM v2 EPDMS. The single-mode anchor alone is weaker than CLOVER, and CLOVER without anchor already obtains 89.3 EPDMS. Using the anchor as a soft reference improves EPDMS to 90.4 and substantially improves EC, supporting its role in reducing frame-to-frame selection jitter among near-tied high-quality candidates. The full anchor weight sweep is provided in Appendix C.

Number of proposals. We also ablate the number of generated trajectories. Using $K = 32, 64$, and 96 proposals yields $93.9, 94.5$, and 94.4 PDMS, respectively. Thus, increasing the proposal set from 32 to 64 improves candidate coverage, while further increasing to 96 brings no additional gain and may slightly increase ranking difficulty; we use $K = 64$ by default.

6 Limitations and Future Work

CLOVER currently performs scoring and ranking mainly at the per-scene trajectory level. While effective for PDMS-style evaluation, temporally aggregated EPDMS terms such as extended comfort depend on cross-frame consistency, which is only mitigated by our optional anchor-assisted reranking. Future work could integrate history-aware or sequence-level scorers to model temporal consistency directly during ranking and refinement.

Table 6: **Main component ablation on NAVSIM v1.** We ablate pseudo-expert diversity supervision and Stage-2 closed-loop refinement. Diversity supervision provides a stronger proposal set, while closed-loop refinement is most effective when applied on top of the expanded proposal distribution.

Setting	Diversity Sup.	Stage-2 Refinement	PDMS $\uparrow$
Single-expert supervision	–	–	93.7
Diversity only	✓	–	94.1
Closed-loop only	–	✓	93.8
CLOVER full	✓	✓	94.5

Table 7: **Ablation of Stage-2 refinement.** Non-iterative joint updating causes severe distribution drift. Self-distillation without scorer fitting remains stable but underperforms the full alternating scorer-generator refinement.

Stage-2 variant	PDMS $\uparrow$
Non-iterative generator/scorer update	< 10
Self-distillation only, 64 trajectories	94.0
Self-distillation only, 32 trajectories	93.8
Self-distillation only, 16 trajectories	93.8
Self-distillation only, 8 trajectories	93.8
Full alternating closed-loop refinement	94.5

7 Conclusion

We presented **CLOVER**, a closed-loop value estimation and ranking framework for proposal-selection based end-to-end autonomous driving planning. CLOVER addresses the training–evaluation mismatch by first expanding single-trajectory imitation into evaluator-filtered set-level proposal coverage, and then refining the generator through conservative closed-loop self-distillation. A trajectory-level scorer is fitted to true evaluator sub-scores on generated proposals, while teacher-selected top- k and vector-Pareto targets guide the generator with stability regularization. We further analyzed when an imperfect scorer can reliably improve proposal generation, showing that conservative refinement can increase high-quality proposal support when scorer-selected targets are statistically enriched under the true evaluator. Experiments on NAVSIM v1, NAVSIM v2 EPDMS, NavHard, and supplementary nuScenes evaluation demonstrate strong planning performance, while proposal-quality analyses show that CLOVER improves not only final top-1 selection but also the underlying candidate distribution.

References

- [1] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuscenes: A multimodal dataset for autonomous driving. In *CVPR*, 2020.
- [2] Wei Cao, Marcel Hallgarten, Tianyu Li, Daniel Dauner, Xunjiang Gu, Caojun Wang, Yakov Miron, Marco Aiello, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, Andreas Geiger, and Kashyap Chitta. Pseudo-simulation for autonomous driving. In *CoRL*, 2025.
- [3] Canyu Chen, Yuguang Yang, Zhewen Tan, Yizhi Wang, Ruiyi Zhan, Haiyan Liu, Xuanyao Mao, Jason Bao, Xinyue Tang, Linlin Yang, Bingchuan Sun, Yan Wang, and Baochang Zhang. Devil is in narrow policy: Unleashing exploration in driving vla models, 2026. URL <https://arxiv.org/abs/2603.06049>.
- [4] Shaoyu Chen, Bo Jiang, Hao Gao, Bencheng Liao, Qing Xu, Qian Zhang, Chang Huang, Wenyu Liu, and Xinggang Wang. Vadv2: End-to-end vectorized autonomous driving via probabilistic planning. *arXiv preprint arXiv:2402.13243*, 2024.
- [5] Yuntao Chen, Yuqi Wang, and Zhaoxiang Zhang. Drivinggpt: Unifying driving world modeling and planning with multi-modal autoregressive transformers. *arXiv preprint arXiv:2412.18607*, 2024.

Table 8: **Ablation of teacher-set construction in Stage 2.** Scalar real-PDMS top- k targets tend to concentrate proposals around a narrow high-score mode. Distance suppression partially restores diversity, while vector-Pareto guidance achieves the best PDMS and is expected to preserve more diverse high-quality proposal modes. Q-Cluster denotes Qualified Cluster Count@2m computed among proposals with PDMS ≥ 0.8 . Top-6 ADE/FDE are computed among the six highest real-PDMS proposals.

Teacher target construction	PDMS $\uparrow$	Q-Cluster@2m $\uparrow$	Top-6 ADE $\uparrow$	Top-6 FDE $\uparrow$
Real-PDMS top- k	93.9	6.45	1.04	2.31
Real-PDMS top- k + distance suppression	94.3	7.49	1.66	2.47
Vector-Pareto targets (ours)	94.5	8.71	1.75	2.59

Table 9: **Anchor-assisted soft reranking ablation on NAVSIM v2 EPDMS.** The anchor model alone is weaker than CLOVER, but using it as a soft reference improves EPDMS by reducing diversity-induced selection jitter.

Inference setting	EC $\uparrow$	EPDMS $\uparrow$
Single-mode anchor only	77.6	88.7
CLOVER without anchor	35.0	89.3
CLOVER + anchor-assisted soft reranking	75.5	90.4

- [6] Haohan Chi, Huan-ang Gao, Ziming Liu, Jianing Liu, Chenyu Liu, Jinwei Li, Kaisen Yang, Yangcheng Yu, Zeda Wang, Wenyi Li, et al. Impromptu vla: Open weights and open data for driving vision-language-action models. *arXiv preprint arXiv:2505.23757*, 2025.
- [7] Kashyap Chitta, Aditya Prakash, Bernhard Jaeger, Zehao Yu, Katrin Renz, and Andreas Geiger. Transfuser: Imitation with transformer-based sensor fusion for autonomous driving. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45(11):12878–12895, 2022.
- [8] Daniel Dauner, Marcel Hallgarten, Andreas Geiger, and Kashyap Chitta. Parting with misconceptions about learning-based vehicle motion planning. In *CoRL*, 2023.
- [9] Daniel Dauner, Marcel Hallgarten, Tianyu Li, Xinshuo Weng, Zhiyu Huang, Zetong Yang, Hongyang Li, Igor Gilitschenski, Boris Ivanovic, Marco Pavone, et al. Navsim: Data-driven non-reactive autonomous vehicle simulation and benchmarking. In *NeurIPS*, 2024.
- [10] Lan Feng, Yang Gao, Eloi Zablocki, Quanyi Li, Wuyang Li, Sichao Liu, Matthieu Cord, and Alexandre Alahi. Rap: 3d rasterization augmented end-to-end planning. *arXiv preprint arXiv:2510.04333*, 2025.
- [11] Renju Feng, Ning Xi, Duanfeng Chu, Rukang Wang, Zejian Deng, Anzheng Wang, Liping Lu, Jinxiang Wang, and Yanjun Huang. Artemis: Autoregressive end-to-end trajectory planning with mixture of experts for autonomous driving. *arXiv preprint arXiv:2504.19580*, 2025.
- [12] Ke Guo, Haochen Liu, Xiaojun Wu, Jia Pan, and Chen Lv. ipad: Iterative proposal-centric end-to-end autonomous driving. *arXiv preprint arXiv:2505.15111*, 2025.
- [13] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. In *ICLR*, 2022.
- [14] Shengchao Hu, Li Chen, Penghao Wu, Hongyang Li, Junchi Yan, and Dacheng Tao. St-p3: End-to-end vision-based autonomous driving via spatial-temporal feature learning. In *European Conference on Computer Vision*, pages 533–549. Springer, 2022.
- [15] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, Lewei Lu, Xiaosong Jia, Qiang Liu, Jifeng Dai, Yu Qiao, and Hongyang Li. Planning-oriented autonomous driving. In *CVPR*, 2023.
- [16] Yihan Hu, Jiazhi Yang, Li Chen, Keyu Li, Chonghao Sima, Xizhou Zhu, Siqi Chai, Senyao Du, Tianwei Lin, Wenhai Wang, et al. Planning-oriented autonomous driving. In *CVPR*, pages 17853–17862, 2023.

- [17] Jyh-Jing Hwang, Runsheng Xu, Hubert Lin, Wei-Chih Hung, Jingwei Ji, Kristy Choi, Di Huang, Tong He, Paul Covington, Benjamin Sapp, et al. Emma: End-to-end multimodal model for autonomous driving. *arXiv preprint arXiv:2410.23262*, 2024.
- [18] Bo Jiang, Shaoyu Chen, Qing Xu, Bencheng Liao, Jiajie Chen, Helong Zhou, Qian Zhang, Wenyu Liu, Chang Huang, and Xinggang Wang. Vad: Vectorized scene representation for efficient autonomous driving. In *ICCV*, 2023.
- [19] Siwen Jiao, Kangan Qian, Hao Ye, Yang Zhong, Ziang Luo, Sicong Jiang, Zilin Huang, Yangyi Fang, Jinyu Miao, Zheng Fu, et al. Evadrive: Evolutionary adversarial policy optimization for end-to-end autonomous driving. *arXiv preprint arXiv:2508.09158*, 2025.
- [20] Ellington Kirby, Alexandre Boulch, Yihong Xu, Yuan Yin, Gilles Puy, Éloi Zablocki, Andrei Bursuc, Spyros Gidaris, Renaud Marlet, Florent Bartoccioni, et al. Driving on registers. *arXiv preprint arXiv:2601.05083*, 2026.
- [21] Kailin Li, Zhenxin Li, Shiyi Lan, Yuan Xie, Zhizhong Zhang, Jiayi Liu, Zuxuan Wu, Zhiding Yu, and Jose M Alvarez. Hydra-mdp++: Advancing end-to-end driving via expert-guided hydra-distillation. *arXiv e-prints*, pages arXiv–2503, 2025.
- [22] Yingyan Li, Shuyao Shang, Weisong Liu, Bing Zhan, Haochen Wang, Yuqi Wang, Yuntao Chen, Xiaoman Wang, Yasong An, Chufeng Tang, et al. Drivevla-w0: World models amplify data scaling law in autonomous driving. *arXiv preprint arXiv:2510.12796*, 2025.
- [23] Yongkang Li, Kaixin Xiong, Xiangyu Guo, Fang Li, Sixu Yan, Gangwei Xu, Lijun Zhou, Long Chen, Haiyang Sun, Bing Wang, et al. Recogdrive: A reinforced cognitive framework for end-to-end autonomous driving. *arXiv preprint arXiv:2506.08052*, 2025.
- [24] Zhenxin Li, Kailin Li, Shihao Wang, Shiyi Lan, Zhiding Yu, Yishen Ji, Zhiqi Li, Ziyue Zhu, Jan Kautz, Zuxuan Wu, et al. Hydra-mdp: End-to-end multimodal planning with multi-target hydra-distillation. *arXiv preprint arXiv:2406.06978*, 2024.
- [25] Zhenxin Li, Wenhao Yao, Zi Wang, Xinglong Sun, Joshua Chen, Nadine Chang, Maying Shen, Zuxuan Wu, Shiyi Lan, and Jose M Alvarez. Generalized trajectory scoring for end-to-end multimodal planning. *arXiv preprint arXiv:2506.06664*, 2025.
- [26] Zhiqi Li, Zhiding Yu, Shiyi Lan, Jiahao Li, Jan Kautz, Tong Lu, and Jose M Alvarez. Is ego status all you need for open-loop end-to-end autonomous driving? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 14864–14873, 2024.
- [27] Bencheng Liao, Shaoyu Chen, Haoran Yin, Bo Jiang, Cheng Wang, Sixu Yan, Xinbang Zhang, Xiangyu Li, Ying Zhang, Qian Zhang, et al. Diffusiondrive: Truncated diffusion model for end-to-end autonomous driving. In *CVPR*, pages 12037–12047, 2025.
- [28] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Mido Assran, Nicolas Ballas, Wojciech Galuba, Russell Howes, Po-Yao Huang, Shang-Wen Li, Ishan Misra, Michael Rabbat, Vasu Sharma, Gabriel Synnaeve, Hu Xu, Hervé Jégou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision. *TMLR*, 2024.
- [29] Chen Shi, Shaoshuai Shi, Kehua Sheng, Bo Zhang, and Li Jiang. Drivex: Omni scene modeling for learning generalizable world knowledge in autonomous driving. In *ICCV*, 2025.
- [30] Chonghao Sima, Kashyap Chitta, Zhiding Yu, Shiyi Lan, Ping Luo, Andreas Geiger, Hongyang Li, and Jose M Alvarez. Centaur: Robust end-to-end autonomous driving with test-time training. *arXiv preprint arXiv:2503.11650*, 2025.
- [31] Ziyang Song, Lin Liu, Hongyu Pan, Bencheng Liao, Mingzhe Guo, Lei Yang, Yongchang Zhang, Shaoqing Xu, Caiyan Jia, and Yadan Luo. Breaking imitation bottlenecks: Reinforced diffusion powers diverse trajectory generation. *arXiv preprint arXiv:2507.04049*, 2025.

- [32] Wenchao Sun, Xuewu Lin, Keyu Chen, Zixiang Pei, Xiang Li, Yining Shi, and Sifa Zheng. Sparsedrivev2: Scoring is all you need for end-to-end autonomous driving, 2026. URL <https://arxiv.org/abs/2603.29163>.
- [33] Yuqi Wang, Xinghang Li, Wenxuan Wang, Junbo Zhang, Yingyan Li, Yuntao Chen, Xinlong Wang, and Zhaoxiang Zhang. Unified vision-language-action model. *arXiv preprint arXiv:2506.19850*, 2025.
- [34] Xinshuo Weng, Boris Ivanovic, Yan Wang, Yue Wang, and Marco Pavone. Para-drive: Parallelized architecture for real-time autonomous driving. In *CVPR*, 2024.
- [35] Maciej K Wozniak, Lianhang Liu, Yixi Cai, and Patric Jensfelt. Prix: Learning to plan from raw pixels for end-to-end autonomous driving. *arXiv preprint arXiv:2507.17596*, 2025.
- [36] Shuo Xing, Chengyuan Qian, Yuping Wang, Hongyuan Hua, Kexin Tian, Yang Zhou, and Zhengzhong Tu. Openemma: Open-source multimodal model for end-to-end autonomous driving. In *Proceedings of the Winter Conference on Applications of Computer Vision*, pages 1001–1009, 2025.
- [37] Zebin Xing, Xingyu Zhang, Yang Hu, Bo Jiang, Tong He, Qian Zhang, Xiaoxiao Long, and Wei Yin. Goalflow: Goal-driven flow matching for multimodal trajectories generation in end-to-end autonomous driving. In *CVPR*, 2025.
- [38] Wenhao Yao, Zhenxin Li, Shiyi Lan, Zi Wang, Xinglong Sun, Jose M Alvarez, and Zuxuan Wu. Drivesuprim: Towards precise trajectory selection for end-to-end planning. *arXiv preprint arXiv:2506.06659*, 2025.
- [39] Chengran Yuan, Zhanqi Zhang, Jiawei Sun, Shuo Sun, Zefan Huang, Christina Dao Wen Lee, Dongen Li, Yuhang Han, Anthony Wong, Keng Peng Tee, et al. Drama: An efficient end-to-end motion planner for autonomous driving with mamba. *arXiv preprint arXiv:2408.03601*, 2024.
- [40] Yupeng Zheng, Pengxuan Yang, Zebin Xing, Qichao Zhang, Yuhang Zheng, Yinfeng Gao, Pengfei Li, Teng Zhang, Zhongpu Xia, Peng Jia, et al. World4drive: End-to-end autonomous driving via intention-aware physical latent world model. In *ICCV*, 2025.
- [41] Xingcheng Zhou, Xuyuan Han, Feng Yang, Yunpu Ma, Volker Tresp, and Alois Knoll. Open-drivevla: Towards end-to-end autonomous driving with large vision language action model. *arXiv preprint arXiv:2503.23463*, 2025.
- [42] Zewei Zhou, Tianhui Cai, Seth Z Zhao, Yun Zhang, Zhiyu Huang, Bolei Zhou, and Jiaqi Ma. Autovla: A vision-language-action model for end-to-end autonomous driving with adaptive reasoning and reinforcement fine-tuning. *arXiv preprint arXiv:2506.13757*, 2025.
- [43] Jialv Zou, Shaoyu Chen, Bencheng Liao, Zhiyu Zheng, Yuehao Song, Lefei Zhang, Qian Zhang, Wenyu Liu, and Xinggang Wang. Diffusiondrivev2: Reinforcement learning-constrained truncated diffusion modeling in end-to-end autonomous driving. *arXiv preprint arXiv:2512.07745*, 2025.

A Pseudo-Expert Trajectory Generation

CLOVER uses pseudo-expert trajectories to provide set-level multi-modal supervision beyond the single logged trajectory. The pseudo-expert pipeline is used only during training. It constructs a diverse trajectory pool using privileged metric-cache information, scores and filters the candidates with the NAVSIM evaluator, and converts the resulting pool into compact per-scene supervision targets during Stage-1 training. The overall data flow is:

```
metric cache → candidate family generation
               → drivable-area / occupancy pre-check
               → PDM scoring
               → coverage-aware selection
               → post-processing
               → training-time pseudo-expert sampling
```

Training-time privileged information. The pseudo-expert generator uses information that is available in the training metric cache but not available to the deployed planner. This includes the current ego state, route centerline, drivable-area map, future obstacle occupancy, and the NAVSIM PDM evaluator. Therefore, pseudo-experts should be understood as privileged training supervision rather than an online planner. At inference time, CLOVER only uses the learned generator and scorer.

Trajectory parameterization. Each candidate trajectory is generated in a centerline-aligned coordinate system. Let d_0 be the current ego progress along the route centerline and let lat_0 be the current lateral offset. A candidate is defined by a longitudinal progress curve d_t and a lateral offset curve lat_t , and then mapped back to Cartesian coordinates:

$$d_t = d_0 + \sum_{k \leq t} v_k \Delta t, \quad (8)$$

$$x_t = x_{\text{cl}}(d_t) - \text{lat}_t \sin h_{\text{cl}}(d_t), \quad y_t = y_{\text{cl}}(d_t) + \text{lat}_t \cos h_{\text{cl}}(d_t), \quad (9)$$

$$\theta_t = h_{\text{cl}}(d_t), \quad (10)$$

where $(x_{\text{cl}}, y_{\text{cl}}, h_{\text{cl}})$ denotes the route centerline pose. For lateral transitions, we use a quintic smooth step:

$$s(r) = 6r^5 - 15r^4 + 10r^3, \quad (11)$$

which has zero first- and second-order derivatives at both endpoints. This produces smoother lateral shifts than linear interpolation and enables different comfort profiles even when the initial and final lateral offsets are similar.

Candidate action families. For each scene, we generate a structured pool of candidates from six interpretable action families, summarized in Table 10. These families are designed to cover both longitudinal and lateral variations that affect PDM sub-scores, including progress, time-to-collision, comfort, and drivable-area compliance. The goal is not to keep only high-score candidates, but to expose the model to a broad range of feasible, near-feasible, and informative low-score behaviors.

Table 10: **Pseudo-expert candidate families.** Each scene first generates a structured pool of candidate trajectories from interpretable longitudinal and lateral action families.

Family	Count	Purpose
Constant-speed lateral transitions	200	Cover smooth and aggressive lateral shifts under different speeds
Off-road boundary cases	12	Populate low-score drivable-area violation regions
Acceleration profiles	18	Vary longitudinal progress and speed profiles
Stop-go behaviors	9	Cover safe but low-progress behaviors
Approach-brake behaviors	10	Create TTC / comfort trade-offs
Overshoot lateral motions	12	Add aggressive lateral dynamics and comfort variation
Total	261	Structured candidate pool per scene

Pre-check before PDM scoring. Running the full PDM evaluator on every raw candidate is expensive. We therefore first perform lightweight geometric pre-checks. For drivable-area compliance, we compute ego-vehicle corners at each future pose and query whether they lie within valid drivable polygons. For collision pre-checking, we compare the future ego footprint with future obstacle occupancy from the metric cache. Each trajectory is assigned one of three coarse labels: feasible, near-feasible, or infeasible. We prioritize feasible and near-feasible candidates for full PDM scoring, while retaining a small subset of infeasible candidates to preserve score coverage. In our implementation, at most 180 candidates per scene are sent to the full evaluator.

PDM scoring and sub-score recording. Each selected candidate is evaluated by the NAVSIM PDM evaluator. Besides the final scalar PDM score, we also store its sub-scores, including no-at-fault collisions, drivable-area compliance, ego progress, time-to-collision, comfort, and driving-direction compliance. These sub-scores are used both for scorer supervision and for coverage-aware pseudo-expert selection.

Coverage-aware selection. A naive top-score selection would often keep many similar high-score trajectories, causing the pseudo-expert set to collapse back to a narrow mode. Instead, we select a compact per-scene set by covering different PDM sub-score patterns and score ranges. For each scored candidate, we define a coverage key:

$$\text{key} = (\text{DAC}, \text{Collision}, \text{TTC}, \text{Comfort}, \text{ProgressBin}), \quad (12)$$

where binary or discrete PDM components are rounded to their categorical values, and ego progress is discretized into progress bins. We then greedily select candidates by minimizing:

$$\text{cost}_i = 10 \cdot \text{cov_count}(\text{key}_i) + \text{bin_count}(\text{score_bin}_i), \quad (13)$$

where cov_count counts how many already selected candidates share the same sub-score key, and bin_count counts how many candidates have been selected from the same total-score bin. This encourages coverage over both sub-score combinations and scalar score ranges. We keep up to 50 trajectories per scene in the pseudo-expert dataset before post-processing.

Post-processing and boundary interpolation. The generated pseudo-expert dataset is further cleaned before training. We remove candidates with wrong driving direction, i.e., trajectories whose driving-direction compliance is less than 1.0. We then identify score-boundary regions where small changes in lateral offset cause a large PDM drop. Specifically, within each family and speed group, adjacent lateral targets are compared; if the PDM score drop exceeds a threshold, we interpolate an additional candidate between the high-score and low-score sides and evaluate it again. These boundary-interpolation samples provide fine-grained supervision near metric discontinuities, where small geometric changes can lead to large score differences.

Training-time pseudo-expert sampling. The stored pseudo-expert pool is not used directly in full during training. For each scene, we first keep trajectories with PDM score above a threshold, then sort them by score and apply greedy farthest-point sampling in trajectory space. This produces at most $M = 8$ pseudo-experts per scene. If the logged human trajectory is not sufficiently covered by the selected pseudo-expert set, it is added to preserve the human driving prior. The selected trajectories are returned to the model as:

$$\text{pseudo_experts} \in \mathbb{R}^{M \times T \times 3}, \quad \text{pseudo_expert_mask} \in \{0, 1\}^M.$$

If no valid pseudo-expert is available for a scene, the system falls back to the logged human trajectory. This ensures that the training schema remains consistent while retaining standard imitation supervision as a safe fallback.

Key hyperparameters. Table 11 summarizes the main hyperparameters used in pseudo-expert generation and training-time sampling.

Qualitative examples. Figure 4 visualizes pseudo-expert trajectory pools on representative scenes. The candidates cover a wide range of lateral offsets, progress profiles, and safety/comfort trade-offs. The visualization also illustrates why pseudo-expert supervision is more informative than local perturbation around the logged trajectory: the generated pool contains both high-score feasible alternatives and lower-score boundary cases that expose the model to metric discontinuities.

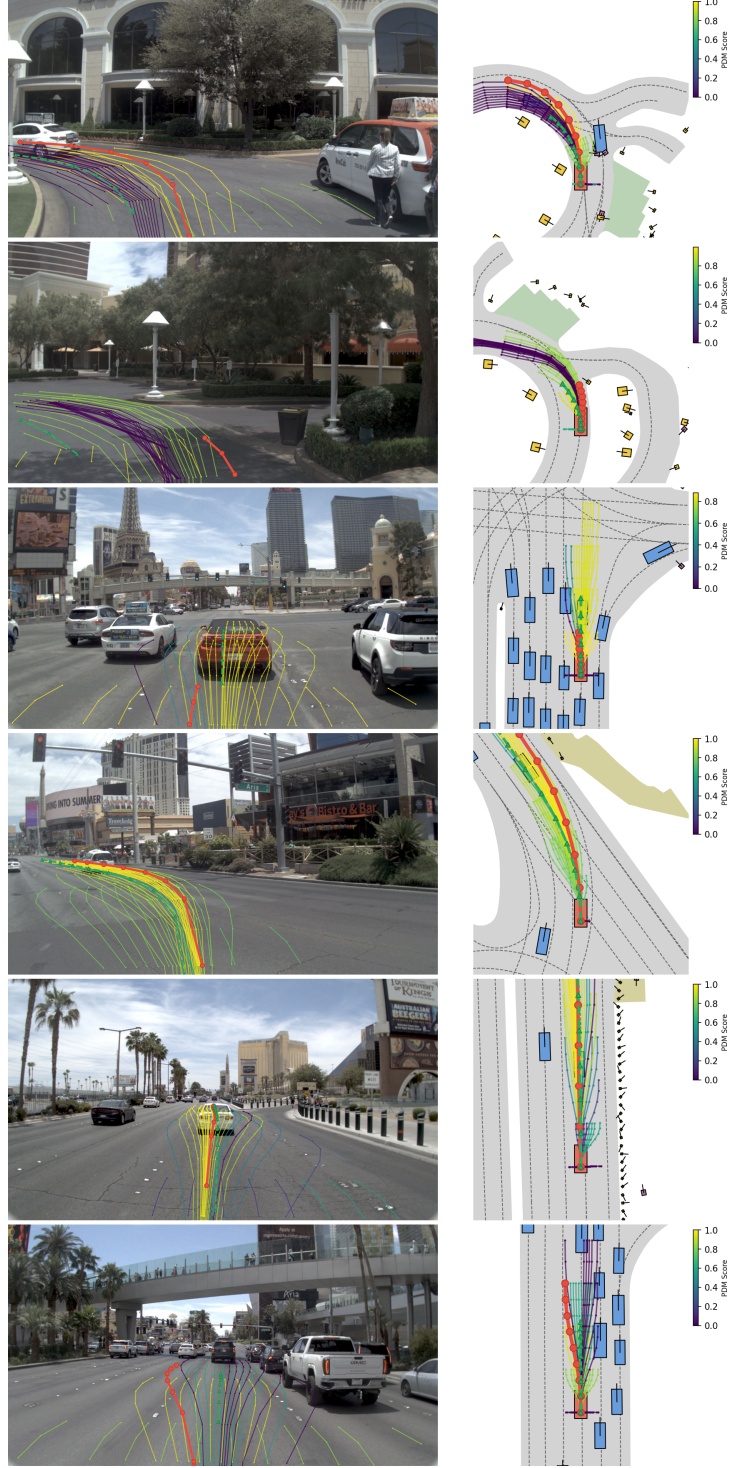


Figure 4: **Visualization of pseudo-expert trajectory candidates.** We show representative pseudo-expert trajectory pools in camera view and bird's-eye view. Candidate trajectories are colored by their true PDM scores. The pseudo-expert generator creates diverse trajectories covering different lateral offsets, progress regimes, and boundary cases. High-score candidates provide feasible multi-modal supervision, while lower-score and boundary candidates help expose the generator and scorer to drivable-area, collision, progress, and comfort variations. These pseudo-experts are used only during Stage-1 training and are not required at inference time.

Table 11: **Key hyperparameters for pseudo-expert generation and sampling.** The candidate pool is generated from structured action families, scored by the evaluator, filtered and selected for coverage, and finally sampled into a compact training-time pseudo-expert set.

Hyperparameter	Value
Number of future poses	8
Sampling interval	0.5 s
Speed candidates	(0, 2, 4, 6, 8, 10, 12, 15) m/s
Regular lateral offsets	(−3.5, −2.0, −1.0, −0.5, 0, 0.5, 1.0, 2.0, 3.5) m
Off-road lateral offsets	(−7.0, −5.5, 5.5, 7.0) m
Transition portions	(0.35, 0.6, 1.0)
Acceleration rates	(−2.0, −1.0, −0.5, 0.5, 1.0, 2.0) m/s ²
Raw candidates per scene	261
Maximum PDM-scored candidates per scene	180
Retained pseudo-expert pool per scene	50
Score-drop threshold for boundary interpolation	0.25
Maximum score boundaries per scene	3
Boundary interpolation samples per boundary	1
Training-time score threshold	0.8
Training-time pseudo-expert top- K	8

Table 12: EPDMS composition in NAVSIM v2 (new metrics relative to v1 are highlighted).

Metric	Weight	Range
No at-fault Collisions (NC)	multiplier	$\{0, \frac{1}{2}, 1\}$
Drivable Area Compliance (DAC)	multiplier	$\{0, 1\}$
Driving Direction Compliance (DDC)	multiplier	$\{0, \frac{1}{2}, 1\}$
Traffic Light Compliance (TLC)	multiplier	$\{0, 1\}$
Ego Progress (EP)	5	$[0, 1]$
Time to Collision (TTC) within bound	5	$\{0, 1\}$
Lane Keeping (LK)	2	$\{0, 1\}$
History Comfort (HC)	2	$\{0, 1\}$
Extended Comfort (EC)	2	$\{0, 1\}$

B Trajectory-quality Score in NAVSIM (PDMS v1 and EPDMS v2)

We use the planning-oriented NAVSIM benchmark and adopt the official Predictive Driver Model Score (PDMS) from NAVSIM v1 as our primary offline trajectory-quality score $s(\cdot, \cdot)$; higher is better. PDMS is a pseudo closed-loop metric that holistically assesses safety, comfort, and progress via multiplicative penalties and a weighted average:

$$\text{PDMS} = \text{NC} \times \text{DAC} \times \left(\frac{5 \cdot \text{EP} + 5 \cdot \text{TTC} + 2 \cdot \text{C}}{12} \right), \quad (14)$$

where NC denotes no at-fault collisions, DAC drivable-area compliance, EP ego progress, TTC time-to-collision within bound, and C comfort.

NAVSIM v2: Extended PDMS (EPDMS). NAVSIM v2 extends PDMS to improve coverage and fairness of open-loop planning evaluation. Compared to NAVSIM v1, EPDMS introduces additional weighted subscores (lane keeping and extended comfort variants), additional multiplier penalties (driving direction compliance and traffic light compliance), and a false-positive penalty filtering scheme.

False-positive penalty filtering. To reduce false-positive penalties, NAVSIM v2 disables a penalty when the human agent is also responsible for the corresponding violation. Formally, for a metric m , define

$$\text{filter}_m(\text{agent}, \text{human}) = \begin{cases} 1.0, & \text{if } m(\text{human}) = 0, \\ m(\text{agent}), & \text{otherwise.} \end{cases} \quad (15)$$

Intuitively, if the human baseline also triggers the violation, the metric is neutralized (set to 1.0) rather than penalizing the planner.

EPDMS definition. With the above filtering, EPDMS is defined as

$$\text{EPDMS} = \left(\prod_{m \in \{\text{NC}, \text{DAC}, \text{DDC}, \text{TLC}\}} \text{filter}_m(\text{agent}, \text{human}) \right) \quad (16)$$

$$\cdot \left(\frac{\sum_{m \in \{\text{TTC}, \text{EP}, \text{HC}, \text{LK}, \text{EC}\}} w_m \cdot \text{filter}_m(\text{agent}, \text{human})}{\sum_{m \in \{\text{TTC}, \text{EP}, \text{HC}, \text{LK}, \text{EC}\}} w_m} \right), \quad (17)$$

where w_m are the weights listed in Table 12.

Pseudo closed-loop aggregation in NAVSIM v2. NAVSIM v1 computes metrics after a 4-second non-reactive simulation rollout (background actors follow recorded futures; ego follows the planned trajectory via a controller). NAVSIM v2 uses a two-stage aggregation to better approximate closed-loop behavior while remaining open-loop: (i) a first-stage score is computed on an initial 4-second scene; (ii) multiple follow-up scenes (precomputed rollouts starting from the same initial scene but with different end states) are also scored, and then aggregated with weights given by a Gaussian kernel based on how close each follow-up scene’s start state is to the submitted planner’s first-stage end state. Finally, the first-stage score and the aggregated second-stage score are multiplied to obtain the final aggregated EPDMS score.

C Anchor-assisted Soft Reranking

This appendix describes the optional anchor-assisted soft reranking used in our NAVSIM v2 EPDMS submission. The motivation is that a strongly diverse proposal set can improve oracle coverage, but may also cause the selected trajectory to vary across adjacent evaluation frames. This can reduce extended comfort in EPDMS. We therefore introduce a lightweight anchor reference to encourage temporally smoother selection, without using future information or evaluator scores at inference time.

Anchor model. The anchor trajectory is produced by a single-mode planner with the same observation inputs and the same base architecture as CLOVER. The anchor model is trained with diversity-related settings disabled and standard single-trajectory imitation retained. It provides a stable reference trajectory, but is not used as an evaluator and does not filter candidates.

Soft reranking formula. For each candidate trajectory τ_k , let

$$\mathbf{p}_k(t) = (x_k(t), y_k(t), \psi_k(t)), \quad t = 1, \dots, T,$$

and let the anchor trajectory be

$$\mathbf{a}(t) = (x_a(t), y_a(t), \psi_a(t)).$$

We define the full-trajectory position RMS:

$$\text{XYRMS}_k = \sqrt{\frac{1}{T} \sum_{t=1}^T [(x_k(t) - x_a(t))^2 + (y_k(t) - y_a(t))^2]}, \quad (18)$$

and the full-trajectory heading RMS:

$$\text{HeadRMS}_k = \sqrt{\frac{1}{T} \sum_{t=1}^T \text{wrap}(\psi_k(t) - \psi_a(t))^2}. \quad (19)$$

The anchor penalty is:

$$\text{Penalty}_k = \lambda_{xy} \frac{\text{XYRMS}_k}{S_{\text{pos}}} + \lambda_{\psi} \frac{\text{HeadRMS}_k}{S_{\psi}}, \quad (20)$$

where $S_{\text{pos}} = 5.0$ and $S_{\psi} = 0.35$. Given the learned scorer score s_k , the final soft reranking score is:

$$Q_k = \lambda_s s_k - \text{Penalty}_k. \quad (21)$$

The selected candidate is:

$$k^* = \arg \max_k Q_k. \quad (22)$$

This is a soft scheme: no hard threshold is used to discard candidates.

Table 13: **Coarse sweep for anchor-assisted soft reranking.** We report representative grid-search results over scorer weight λ_s , position-anchor weight λ_{xy} , and heading-anchor weight λ_ψ . Scores are normalized EPDMS values. The no-anchor baseline obtains 0.8930 EPDMS, while a broad range of anchor configurations consistently improves performance. The main result uses $\lambda_s = 2.00$, $\lambda_{xy} = 0.20$, and $\lambda_\psi = 0.50$.

λ_s	λ_{xy}	λ_ψ	Score	EP	EC	HC
–	–	–	0.89322	0.92408	0.3500	0.96459
2.00	0.20	0.50	0.90356	0.8689	0.7550	0.9823
1.75	0.20	0.75	0.90213	0.8613	0.7641	0.9825
2.00	0.20	0.75	0.90292	0.8652	0.7594	0.9825
2.25	0.20	0.75	0.90222	0.8689	0.7509	0.9823
1.75	0.20	1.00	0.90212	0.8596	0.7667	0.9825
2.00	0.20	1.00	0.90212	0.8630	0.7605	0.9825
2.25	0.20	1.00	0.90168	0.8663	0.7539	0.9824
2.00	0.25	0.50	0.90285	0.8633	0.7610	0.9825
2.00	0.25	0.75	0.90194	0.8601	0.7640	0.9825
1.75	0.35	0.75	0.90086	0.8502	0.7698	0.9828
2.00	0.35	0.75	0.90177	0.8534	0.7679	0.9826
2.25	0.35	0.75	0.90216	0.8564	0.7660	0.9825
1.75	0.35	1.00	0.90021	0.8492	0.7718	0.9828
2.00	0.35	1.00	0.90118	0.8521	0.7699	0.9826
2.25	0.35	1.00	0.90180	0.8549	0.7679	0.9825

Parameter search. We sweep the scorer weight λ_s , position-anchor weight λ_{xy} , and heading-anchor weight λ_ψ on the validation/evaluation split. The main reported result uses:

$$\lambda_s = 2.00, \quad \lambda_{xy} = 0.20, \quad \lambda_\psi = 0.50.$$

This configuration achieves the best score in our search, with a final score of 0.90356.

The search shows a consistent trade-off between final score and extended comfort. A light position anchor, around $\lambda_{xy} = 0.20$, preserves progress and achieves the best total score, while stronger heading or position anchoring can improve comfort at the cost of progress and final score.

D Theoretical Analysis

This appendix provides full proofs for the theoretical results in Section 4. All scores are normalized to $[0, 1]$. For simplicity, we write expectations over $o \sim \mathcal{D}$ implicitly when no ambiguity arises.

D.1 Notation

Let $o \sim \mathcal{D}$ be a logged driving scene and let $\tau \in \mathcal{X}_o$ be a candidate ego trajectory. The true non-differentiable trajectory-quality score is

$$R^*(o, \tau) \in [0, 1]. \quad (23)$$

The learned scalar scorer at iteration t is

$$s_t(o, \tau) = \Gamma_{\text{PDMS}}(S_t(o, \tau)) \in [0, 1], \quad (24)$$

where $S_t(o, \tau)$ predicts PDMS-related sub-scores. The generator induces a trajectory distribution $\pi_t(\tau \mid o)$. The true expected score and scorer-surrogate expected score are

$$J^*(\pi) = \mathbb{E}_{o \sim \mathcal{D}, \tau \sim \pi(\cdot \mid o)}[R^*(o, \tau)], \quad (25)$$

$$J_{s_t}(\pi) = \mathbb{E}_{o \sim \mathcal{D}, \tau \sim \pi(\cdot \mid o)}[s_t(o, \tau)]. \quad (26)$$

For a K -proposal set, we also use the empirical distribution

$$\mu_t^o = \frac{1}{K} \sum_{i=1}^K \delta_{\tau_i^t(o)}. \quad (27)$$

D.2 Auxiliary Result: Approximate Monotonicity

We first restate the assumptions. Let the trust region around the current generator be

$$\mathcal{B}_t(\rho) = \{\pi : d(\pi, \pi_t) \leq \rho\}. \quad (28)$$

Assume local scorer accuracy:

$$|J_{s_t}(\pi) - J^*(\pi)| \leq \epsilon_t(\rho), \quad \forall \pi \in \mathcal{B}_t(\rho). \quad (29)$$

Define

$$e_t(\pi) = |J_{s_t}(\pi) - J^*(\pi)|, \quad (30)$$

and the surrogate improvement

$$m_t = J_{s_t}(\pi_{t+1}) - J_{s_t}(\pi_t). \quad (31)$$

By the definition of $e_t(\pi)$,

$$J^*(\pi_{t+1}) \geq J_{s_t}(\pi_{t+1}) - e_t(\pi_{t+1}), \quad (32)$$

and

$$J^*(\pi_t) \leq J_{s_t}(\pi_t) + e_t(\pi_t). \quad (33)$$

Subtracting the second inequality from the first gives

$$J^*(\pi_{t+1}) - J^*(\pi_t) \geq J_{s_t}(\pi_{t+1}) - J_{s_t}(\pi_t) - e_t(\pi_{t+1}) - e_t(\pi_t) \quad (34)$$

$$= m_t - e_t(\pi_{t+1}) - e_t(\pi_t). \quad (35)$$

This proves the general form. Therefore, if

$$m_t \geq e_t(\pi_{t+1}) + e_t(\pi_t), \quad (36)$$

then

$$J^*(\pi_{t+1}) \geq J^*(\pi_t). \quad (37)$$

Now assume the generator update is conservative and remains in $\mathcal{B}_t(\rho)$. If the generator phase improves the regularized surrogate up to residual $\alpha_t \geq 0$:

$$J_{s_t}(\pi_{t+1}) - \lambda D(\pi_{t+1}, \pi_t) \geq J_{s_t}(\pi_t) - \alpha_t, \quad (38)$$

with $D(\pi_{t+1}, \pi_t) \geq 0$, then

$$J_{s_t}(\pi_{t+1}) \geq J_{s_t}(\pi_t) - \alpha_t, \quad (39)$$

so $m_t \geq -\alpha_t$. Since $\pi_t, \pi_{t+1} \in \mathcal{B}_t(\rho)$,

$$e_t(\pi_t) \leq \epsilon_t(\rho), \quad e_t(\pi_{t+1}) \leq \epsilon_t(\rho). \quad (40)$$

Substituting into the general bound yields

$$J^*(\pi_{t+1}) \geq J^*(\pi_t) - 2\epsilon_t(\rho) - \alpha_t. \quad (41)$$

This completes the proof.

D.3 Auxiliary Result: One-step Scorer Extrapolation

Define the scorer error function

$$g_t(o, \tau) = |s_t(o, \tau) - R^*(o, \tau)|. \quad (42)$$

Since both s_t and R^* lie in $[0, 1]$, we have $0 \leq g_t \leq 1$. Assume that the scorer phase fits the current generator distribution:

$$\mathbb{E}_{\tau \sim \pi_t}[g_t(o, \tau)] \leq \epsilon_t. \quad (43)$$

Let the one-step drift be

$$\Delta_t = \text{TV}(\pi_{t+1}, \pi_t). \quad (44)$$

For any function $f \in [0, 1]$,

$$|\mathbb{E}_P[f] - \mathbb{E}_Q[f]| \leq \text{TV}(P, Q). \quad (45)$$

Thus,

$$\mathbb{E}_{\tau \sim \pi_{t+1}}[g_t(o, \tau)] \leq \mathbb{E}_{\tau \sim \pi_t}[g_t(o, \tau)] + \text{TV}(\pi_{t+1}, \pi_t) \quad (46)$$

$$\leq \epsilon_t + \Delta_t. \quad (47)$$

Summing over $t = 0, \dots, T-1$ gives

$$\sum_{t=0}^{T-1} \mathbb{E}_{\pi_{t+1}}[|s_t - R^*|] \leq \sum_{t=0}^{T-1} \epsilon_t + \sum_{t=0}^{T-1} \Delta_t. \quad (48)$$

If $\epsilon_t \leq \epsilon$ and $\Delta_t \leq \rho$, the bound becomes

$$\sum_{t=0}^{T-1} \mathbb{E}_{\pi_{t+1}}[|s_t - R^*|] \leq T\epsilon + T\rho. \quad (49)$$

We now compare this with using a fixed initial scorer s_0 . If

$$\mathbb{E}_{\pi_0}[|s_0 - R^*|] \leq \epsilon_0, \quad (50)$$

then

$$\mathbb{E}_{\pi_{t+1}}[|s_0 - R^*|] \leq \epsilon_0 + \text{TV}(\pi_{t+1}, \pi_0). \quad (51)$$

By the triangle inequality,

$$\text{TV}(\pi_{t+1}, \pi_0) \leq \sum_{k=0}^t \Delta_k. \quad (52)$$

Therefore,

$$\mathbb{E}_{\pi_{t+1}}[|s_0 - R^*|] \leq \epsilon_0 + \sum_{k=0}^t \Delta_k. \quad (53)$$

Summing over t yields

$$\sum_{t=0}^{T-1} \mathbb{E}_{\pi_{t+1}}[|s_0 - R^*|] \leq T\epsilon_0 + \sum_{t=0}^{T-1} \sum_{k=0}^t \Delta_k \quad (54)$$

$$= T\epsilon_0 + \sum_{k=0}^{T-1} (T-k)\Delta_k. \quad (55)$$

If $\Delta_k \leq \rho$, this is

$$T\epsilon_0 + \frac{T(T+1)}{2}\rho. \quad (56)$$

Thus, refitting the scorer reduces the drift dependence from quadratic to linear in the number of refinement steps.

D.4 Auxiliary Result: Approximate Pareto Consistency

Let the true vector score be

$$\mathbf{R}^*(o, \tau) = (R_1^*(o, \tau), \dots, R_M^*(o, \tau)), \quad (57)$$

and the predicted vector score be

$$\mathbf{s}_t(o, \tau) = (s_{t,1}(o, \tau), \dots, s_{t,M}(o, \tau)). \quad (58)$$

Assume uniform vector error on the candidate pool:

$$\|\mathbf{s}_t(o, \tau) - \mathbf{R}^*(o, \tau)\|_\infty \leq \delta_t, \quad \forall \tau \in \mathcal{C}_t(o). \quad (59)$$

We define the true ϵ -Pareto set as

$$\mathcal{P}_\epsilon^*(o) = \{\tau \in \mathcal{C}_t(o) : \nexists \tau' \in \mathcal{C}_t(o) \text{ such that } R_m^*(o, \tau') \geq R_m^*(o, \tau) + \epsilon, \forall m\}. \quad (60)$$

The scorer-induced Pareto set is

$$\mathcal{P}_{s_t}(o) = \{\tau \in \mathcal{C}_t(o) : \nexists \tau' \in \mathcal{C}_t(o) \text{ such that } s_{t,m}(o, \tau') \geq s_{t,m}(o, \tau), \forall m, \text{ and } \exists m : s_{t,m}(o, \tau') > s_{t,m}(o, \tau)\}. \quad (61)$$

We prove that for any $\kappa > 0$,

$$\mathcal{P}_{s_t}(o) \subseteq \mathcal{P}_{2\delta_t + \kappa}^*(o). \quad (62)$$

Suppose, for contradiction, that there exists $\tau \in \mathcal{P}_{s_t}(o)$ but $\tau \notin \mathcal{P}_{2\delta_t + \kappa}^*(o)$. Then there exists $\tau' \in \mathcal{C}_t(o)$ such that

$$R_m^*(o, \tau') \geq R_m^*(o, \tau) + 2\delta_t + \kappa, \quad \forall m. \quad (63)$$

By the error bound,

$$s_{t,m}(o, \tau') \geq R_m^*(o, \tau') - \delta_t, \quad (64)$$

and

$$s_{t,m}(o, \tau) \leq R_m^*(o, \tau) + \delta_t. \quad (65)$$

Combining these inequalities gives

$$s_{t,m}(o, \tau') \geq R_m^*(o, \tau) + 2\delta_t + \kappa - \delta_t \quad (66)$$

$$= R_m^*(o, \tau) + \delta_t + \kappa \quad (67)$$

$$\geq s_{t,m}(o, \tau) + \kappa. \quad (68)$$

Thus,

$$s_{t,m}(o, \tau') > s_{t,m}(o, \tau), \quad \forall m. \quad (69)$$

Therefore τ' strictly dominates τ in scorer space, contradicting $\tau \in \mathcal{P}_{s_t}(o)$. This proves the result.

The additional $\kappa > 0$ only excludes the degenerate boundary case where the true margin is exactly $2\delta_t$ and the scorer errors simultaneously attain their worst-case values in every objective. Since κ can be arbitrarily small, the result can be interpreted as containment in the right-limit $(2\delta_t)^+$ -approximate Pareto region.

Coverage-error extension. Assume that the student generator covers each predicted Pareto target within distance η_t : for any $\tau \in \mathcal{P}_{s_t}(o)$, there exists a student proposal $\tilde{\tau}$ such that

$$d_{\mathcal{X}}(\tilde{\tau}, \tau) \leq \eta_t. \quad (70)$$

Assume also that $\mathbf{R}^*$ is L -Lipschitz on the valid proposal manifold:

$$\|\mathbf{R}^*(o, \tau) - \mathbf{R}^*(o, \tau')\|_{\infty} \leq L d_{\mathcal{X}}(\tau, \tau'). \quad (71)$$

Then the covered student proposal lies in the true $(2\delta_t + \kappa + L\eta_t)$ -approximate Pareto region. Indeed, if $\tilde{\tau}$ were dominated by some τ' with margin $2\delta_t + \kappa + L\eta_t$, then

$$R_m^*(o, \tau') \geq R_m^*(o, \tilde{\tau}) + 2\delta_t + \kappa + L\eta_t. \quad (72)$$

Since

$$R_m^*(o, \tilde{\tau}) \geq R_m^*(o, \tau) - L\eta_t, \quad (73)$$

we obtain

$$R_m^*(o, \tau') \geq R_m^*(o, \tau) + 2\delta_t + \kappa, \quad \forall m, \quad (74)$$

which contradicts $\tau \in \mathcal{P}_{2\delta_t + \kappa}^*(o)$.

D.5 Proof of Theorem 1

For a scene o , define the true high-score region

$$\mathcal{H}_o = \{\tau : R^*(o, \tau) \geq r_{\text{high}}\}. \quad (75)$$

Let the current proposal distribution be

$$\mu_t^o = \frac{1}{K} \sum_{i=1}^K \delta_{\tau_i^t(o)}. \quad (76)$$

Let $A_t(o)$ denote the scorer-selected top- k or vector-Pareto target set, and let

$$\nu_t^o = \frac{1}{|A_t(o)|} \sum_{\tau \in A_t(o)} \delta_\tau \quad (77)$$

be the empirical distribution on this target set. Define

$$p_t(o) = \mu_t^o(\mathcal{H}_o), \quad q_t(o) = \nu_t^o(\mathcal{H}_o). \quad (78)$$

The selected-set enrichment assumption states that

$$q_t(o) \geq p_t(o) + \xi_t(o), \quad \xi_t(o) > 0. \quad (79)$$

The conservative refinement assumption states that the actual student distribution is close to the ideal mixture

$$\bar{\mu}_{t+1}^o = (1 - \alpha_t)\mu_t^o + \alpha_t\nu_t^o, \quad \alpha_t \in [0, 1], \quad (80)$$

in total variation:

$$\text{TV}(\mu_{t+1}^o, \bar{\mu}_{t+1}^o) \leq \eta_t(o). \quad (81)$$

Then

$$\bar{\mu}_{t+1}^o(\mathcal{H}_o) = (1 - \alpha_t)\mu_t^o(\mathcal{H}_o) + \alpha_t\nu_t^o(\mathcal{H}_o) \quad (82)$$

$$= (1 - \alpha_t)p_t(o) + \alpha_tq_t(o) \quad (83)$$

$$\geq (1 - \alpha_t)p_t(o) + \alpha_t[p_t(o) + \xi_t(o)] \quad (84)$$

$$= p_t(o) + \alpha_t\xi_t(o). \quad (85)$$

For any event A , total variation implies

$$\mu_{t+1}^o(A) \geq \bar{\mu}_{t+1}^o(A) - \eta_t(o). \quad (86)$$

Taking $A = \mathcal{H}_o$ yields

$$p_{t+1}(o) = \mu_{t+1}^o(\mathcal{H}_o) \geq p_t(o) + \alpha_t\xi_t(o) - \eta_t(o). \quad (87)$$

This proves the high-score support statement.

Expected-score version. Assume instead that the selected target distribution has higher true expected score:

$$\mathbb{E}_{\nu_t^o}[R^*] - \mathbb{E}_{\mu_t^o}[R^*] \geq \beta_t(o) > 0. \quad (88)$$

Then

$$\mathbb{E}_{\bar{\mu}_{t+1}^o}[R^*] = (1 - \alpha_t)\mathbb{E}_{\mu_t^o}[R^*] + \alpha_t\mathbb{E}_{\nu_t^o}[R^*] \quad (89)$$

$$= \mathbb{E}_{\mu_t^o}[R^*] + \alpha_t(\mathbb{E}_{\nu_t^o}[R^*] - \mathbb{E}_{\mu_t^o}[R^*]) \quad (90)$$

$$\geq \mathbb{E}_{\mu_t^o}[R^*] + \alpha_t\beta_t(o). \quad (91)$$

Because $R^* \in [0, 1]$ and $\text{TV}(\mu_{t+1}^o, \bar{\mu}_{t+1}^o) \leq \eta_t(o)$,

$$\mathbb{E}_{\mu_{t+1}^o}[R^*] \geq \mathbb{E}_{\bar{\mu}_{t+1}^o}[R^*] - \eta_t(o). \quad (92)$$

Thus,

$$\mathbb{E}_{\mu_{t+1}^o}[R^*] \geq \mathbb{E}_{\mu_t^o}[R^*] + \alpha_t\beta_t(o) - \eta_t(o). \quad (93)$$

Multi-round extension. Summing over $t = 0, \dots, T-1$ gives

$$\mathbb{E}_{\mu_T^o}[R^*] \geq \mathbb{E}_{\mu_0^o}[R^*] + \sum_{t=0}^{T-1} \alpha_t\beta_t(o) - \sum_{t=0}^{T-1} \eta_t(o). \quad (94)$$

Therefore, Stage-2 refinement improves the true expected proposal quality whenever

$$\sum_{t=0}^{T-1} \alpha_t\beta_t(o) > \sum_{t=0}^{T-1} \eta_t(o). \quad (95)$$

From high-score support to Oracle@K. If K proposals are interpreted as samples from μ_t^o , the probability that at least one proposal falls in the high-score region is

$$A_K(p_t) = 1 - (1 - p_t)^K. \quad (96)$$

If all trajectories in $\mathcal{H}_o$ have score at least r_{high} and all candidates have score at least r_{min} , then the oracle score satisfies

$$O_t(o) \geq r_{\text{min}} + (r_{\text{high}} - r_{\text{min}})A_K(p_t). \quad (97)$$

Since $A_K(p)$ is monotone in p , increasing $p_t(o)$ increases this lower bound on Oracle@K.

From Oracle@K to deployed top-1. Let

$$O_t(o) = \max_{\tau \in T_t(o)} R^*(o, \tau) \quad (98)$$

be the true oracle in the proposal set, and let

$$V_t(o) = R^* \left(o, \arg \max_{\tau \in T_t(o)} s_t(o, \tau) \right) \quad (99)$$

be the true score of the deployed scorer-selected top-1. Define ranking regret

$$G_t(o) = O_t(o) - V_t(o). \quad (100)$$

Then

$$V_T(o) - V_0(o) = [O_T(o) - O_0(o)] + [G_0(o) - G_T(o)]. \quad (101)$$

Thus, deployed top-1 improvement comes from two sources: improving the proposal oracle and reducing ranking regret.

D.6 Uniform Margin as a Sufficient Condition

The selected-set enrichment assumption in Theorem 1 is weaker than worst-case scorer calibration. A clean sufficient condition is the standard margin condition.

Assume there are high-score and low-score regions $\mathcal{H}_o$ and $\mathcal{L}_o$ such that

$$R^*(o, \tau_h) \geq r_{\text{high}}, \quad \forall \tau_h \in \mathcal{H}_o, \quad (102)$$

$$R^*(o, \tau_l) \leq r_{\text{low}}, \quad \forall \tau_l \in \mathcal{L}_o, \quad (103)$$

with margin

$$\gamma = r_{\text{high}} - r_{\text{low}} > 0. \quad (104)$$

If the scorer satisfies the uniform error bound

$$|s_t(o, \tau) - R^*(o, \tau)| \leq \epsilon, \quad \forall \tau \in \mathcal{H}_o \cup \mathcal{L}_o, \quad (105)$$

and $\gamma > 2\epsilon$, then for any $\tau_h \in \mathcal{H}_o$ and $\tau_l \in \mathcal{L}_o$,

$$s_t(o, \tau_h) \geq R^*(o, \tau_h) - \epsilon \geq r_{\text{high}} - \epsilon, \quad (106)$$

$$s_t(o, \tau_l) \leq R^*(o, \tau_l) + \epsilon \leq r_{\text{low}} + \epsilon. \quad (107)$$

Therefore,

$$s_t(o, \tau_h) - s_t(o, \tau_l) \geq \gamma - 2\epsilon > 0. \quad (108)$$

Thus high-score proposals are ranked above low-score proposals.

This condition is sufficient but not necessary. In practice, the worst-case error ϵ_{max} can be overly conservative. Stage-2 distillation only requires the weaker statistical condition that scorer-selected top- k or Pareto targets are enriched with high true evaluator scores.

Table 14: **Empirical validation of selected-set enrichment.** Worst-case scorer calibration is too conservative, while high-probability separability, high-score precision, and scorer Top- k coverage support the enrichment condition required by Stage-2 distillation.

Validation target	Metric / protocol	Result	Interpretation
Stage-1 support	Pooled full-score proposals	35.42%	Stage 1 provides non-zero high-score support.
Worst-case margin	$P(\gamma_o > 2\epsilon_{\max,o})$, elite/reject 0.05/0.05	10.64%	Worst-case calibration is too strict.
High-prob. margin	$P(\gamma_o > 2\epsilon_{p75,o})$, elite/reject 0.01/0.01	92.50%	Quantile-level separation is widely satisfied.
High-prob. margin	$P(\gamma_o > 2\epsilon_{p90,o})$, elite/reject 0.01/0.01	82.37%	Most scenes satisfy high-probability separation.
High-prob. margin	$P(\gamma_o > 2\epsilon_{p95,o})$, elite/reject 0.01/0.01	71.56%	Strict quantile-level separation still often holds.
Pairwise separability	high ≥ 0.95 , low ≤ 0.50	94.72%	Scorer usually ranks true high-score proposals above low-score ones.
High-score precision	$s \geq 0.90$	mean $R^* = 0.9529$, $P(R^* \geq 0.90) = 90.16\%$	Scorer-high proposals are reliable true high-score candidates.
High-score precision	$s \geq 0.95$	mean $R^* = 0.9753$, $P(R^* \geq 0.90) = 96.69\%$, $P(R^* = 1) = 69.74\%$	High-confidence scorer targets are strongly enriched.
Top- k quality	scorer Top-1	mean best $R^* = 0.9448$, oracle gap 0.0495	Single top selection is already high quality.
Top- k quality	scorer Top-8	mean best $R^* = 0.9656$, oracle gap 0.0283	Top- k targets improve high-score coverage.
Top- k quality	scorer Top-16	mean best $R^* = 0.9730$, oracle gap 0.0209	Larger target sets further reduce oracle gap.

D.7 Empirical Support for Selected-Set Enrichment

We empirically validate the selected-set enrichment assumption used in Theorem 1. The analysis is conducted on the NAVSIM evaluation PKL with 12,146 scenes and 64 proposals per scene.

Table 14 summarizes the empirical proxies. The worst-case margin condition $\gamma > 2\epsilon_{\max}$ is intentionally conservative and is not expected to hold for all proposals. Indeed, under the per-scene elite/reject (0.05/0.05) protocol, only 10.64% of scenes satisfy $\gamma_o > 2\epsilon_{\max,o}$. However, the weaker statistical conditions needed by Stage-2 distillation are strongly supported. Under elite/reject (0.01/0.01), 92.50%, 82.37%, and 71.56% of scenes satisfy $\gamma_o > 2\epsilon_{p75,o}$, $\gamma_o > 2\epsilon_{p90,o}$, and $\gamma_o > 2\epsilon_{p95,o}$, respectively. Moreover, fixed-threshold high/low groups are highly separable: for high ≥ 0.95 and low ≤ 0.50 , pairwise ranking accuracy reaches 94.72%. Most importantly, scorer-high proposals are truly high scoring: when $s \geq 0.95$, the true mean score is 0.9753, $P(R^* \geq 0.90) = 96.69\%$, and $P(R^* = 1) = 69.74\%$, compared with a pooled full-score rate of 35.42%. These observations support the key premise of Theorem 1: Stage 2 improves over Stage 1 not by discovering high-quality modes from scratch, but by reallocating proposal mass toward scorer-selected targets that are statistically enriched with high true evaluator scores.

At the pooled level, let $\mathcal{H} = \{\tau : R^*(\tau) = 1\}$. The proposal pool has

$$p \approx 35.42\%. \quad (109)$$

Using $s \geq 0.95$ as a high-confidence scorer-selected proxy gives

$$q \approx 69.74\%. \quad (110)$$

Thus the enrichment gap is approximately

$$q - p \approx 69.74\% - 35.42\% = 34.32\%. \quad (111)$$

This directly supports the condition $q_t(o) \geq p_t(o) + \xi_t(o)$ in an aggregate sense.

E Implementation Details and Hyperparameters

This appendix summarizes the main implementation details and hyperparameters used in CLOVER. Unless otherwise specified, all models and ablations use the same DrivoR-style base architecture shown in Table 15. Stage-1 diversity pretraining and its loss weights are summarized in Table 16. The pseudo-expert generation configuration used for evaluator-filtered trajectory coverage is provided in Table 17. Stage-2 alternating self-distillation and its teacher-set configuration are summarized in Table 18. Finally, Table 19 reports the compute resources and wall-clock training time.

F Fixed-Proposal Scorer Diagnostic

The proposal diversity analysis shows that CLOVER substantially improves both proposal quality and proposal diversity. However, it also reveals a new bottleneck: once many high-quality candidates are available, final performance depends heavily on whether the scorer can correctly rank near-tied but behaviorally different trajectories. A better scorer can improve the final selected trajectory directly, and in our framework it can also serve as a differentiable mediator for refining the generator during self-distillation. This motivates a standardized diagnostic protocol that focuses specifically on trajectory-level scorer design.

Table 15: **Base DrivoR-style architecture used by CLOVER.** Unless otherwise specified, all models and ablations use this common architecture.

Hyperparameter	Value
Input cameras	front, left, right, back
LiDAR input	disabled
Backbone	DINOv2 ViT-S
Backbone identifier	timm/vit_small_patch14_reg4_dinov2.lvd142m
LoRA	enabled, rank 32
Scene tokens per camera	16
Prediction horizon	4.0 s
Sampling interval	0.5 s
Number of future poses	8
Number of proposals	64
Transformer hidden dimension	256
Transformer FFN dimension	1024
Trajectory representation	(x, y, θ)
One token per trajectory	true
NAVSIM v1 deployment weights	NC/DAC/DDC/TTC/EP/Comfort = 1/1/0/5/5/2

Table 16: **Stage-1 diversity pretraining hyperparameters.** Stage 1 trains the generator with logged-trajectory imitation and evaluator-filtered pseudo-expert coverage, while pretraining the scorer with evaluator-provided trajectory scores.

Hyperparameter	Value
Training epochs	25
Batch size	32
Number of GPUs	4
DataLoader workers	4
Optimizer	AdamW
Base learning rate	2×10^{-4}
Precision	16-mixed
Seed	2
Ray scoring	enabled
Ray threads per node	96
Pseudo-expert top- K	8
Pseudo-expert score threshold	0.8
Pseudo-expert loss weight	0.5
Pseudo-expert applied to all refinement layers	false
Trajectory imitation loss weight	1.0
Final score loss weight	1.0
Prediction cross-entropy loss weight	1.0
Prediction L1 loss weight	0.1
Prediction area loss weight	2.0

We introduce a fixed-proposal scorer diagnostic protocol under a shared CLOVER-generated candidate pool. For each scene in NAVSIM `navtrain` and `navtest`, we use the trained CLOVER generator to produce 64 candidate trajectories. Each candidate is then evaluated by the true PDMS evaluator to obtain scalar scores and interpretable sub-score labels. This produces a scorer-only dataset:

$$\mathcal{D}_{\text{score}} = \left\{ (o, \{\tau_i\}_{i=1}^{64}, \{\mathbf{r}_i^{\text{true}}\}_{i=1}^{64}) \right\}. \quad (112)$$

A scorer architecture is trained to predict sub-scores for each fixed candidate and selects the trajectory with the highest composed score:

$$i_{\phi}^* = \arg \max_i \Gamma_{\text{PDMS}}(S_{\phi}(o, \tau_i)). \quad (113)$$

We report the true PDMS of the selected trajectory:

$$\text{Score} = \frac{1}{|\mathcal{D}|} \sum_{o \in \mathcal{D}} s_{i_{\phi}^*}^{\text{true}}. \quad (114)$$

Table 17: **Pseudo-expert trajectory generation hyperparameters.** Pseudo-expert trajectories are generated from interpretable candidate families and filtered by evaluator-based trajectory quality.

Hyperparameter	Value
Number of poses	8
Sampling interval	0.5 s
Speed candidates	(0, 2, 4, 6, 8, 10, 12, 15) m/s
Regular lateral offsets	(−3.5, −2.0, −1.0, −0.5, 0, 0.5, 1.0, 2.0, 3.5) m
Off-road lateral offsets	(−7.0, −5.5, 5.5, 7.0) m
Transition portions	(0.35, 0.6, 1.0)
Acceleration rates	(−2.0, −1.0, −0.5, 0.5, 1.0, 2.0)
Score bins	(0.0, 0.2, 0.4, 0.6, 0.8, 1.01)
Progress bins	(0.0, 0.2, 0.5, 0.8, 1.01)
Retained trajectories per scene	50
Maximum scored candidates per scene	180

Table 18: **Stage-2 alternating self-distillation hyperparameters.** Stage 2 alternates between scorer fitting and generator refinement. The visual backbone is frozen during Stage 2, while the scorer remains trainable.

Hyperparameter	Value
Initialization	Stage-1 checkpoint
Number of cycles	30
Scorer-fitting epochs per cycle	1
Generator-refinement epochs per cycle	1
Total epochs	60
Batch size	32
Number of GPUs	4
DataLoader workers	8
Base learning rate	3×10^{-5}
Starting phase	scorer fitting
Freeze visual backbone	true
Detach proposals in scorer	true
Sanitize proposals	true
Proposal XY limit	100.0
Ray scoring	enabled
Ray threads per node	96
Trajectory loss weight	0.1
Intermediate loss weight	0.02
Final score loss weight	1.0
Pareto guidance weight	1.0
Teacher stability weight	0.05
Pareto set maximum size	8
Pareto set minimum size	2
Teacher reward threshold	0.4

This protocol isolates scorer quality from proposal generation, because all scorer architectures are evaluated under the same candidate trajectories and the same true labels. It is lightweight, reproducible, and directly targets the ranking component of proposal-selection planners. We deliberately treat it as a scorer diagnostic rather than a full planner benchmark: it does not include generator-scorer closed-loop training, and the fixed offline candidate pool differs from a fully deployed CLOVER system. Therefore, its scores should not be compared directly with full-system PDMS results.

Table 20 compares several scorer backbones under this protocol. DINOv2-Small is the backbone used in our main CLOVER model. Larger DINO backbones provide modest test improvements, while the Wan2.2-5B video-generation backbone achieves the highest diagnostic test score. For Wan2.2-5B, we LoRA fine-tune the video model on NAVSIM and use intermediate block-15 features for scoring. This result is consistent with the intuition that video or world-model features may better capture future-scene-dependent factors required by PDMS. However, Wan2.2-5B also shows a clear train-test gap and is far more expensive than the DINOv2-Small scorer. Similarly, DINO base variants increase

Table 19: **Compute resources and training time.** Training time includes real-time trajectory scoring with the NAVSIM evaluator, which depends on CPU throughput in addition to GPU training speed.

Item	Value
GPUs	4 NVIDIA A100
CPU	AMD EPYC 7742 64-Core Processor
Stage-1 training time	30–40 hours
Stage-2 training time	60–80 hours
Total training time	4–5 days

Table 20: **Fixed-proposal scorer diagnostic.** All scorers are trained and evaluated on the same CLOVER-generated 64-candidate pools with true PDMS sub-score labels. Train and test scores denote the true PDMS of the scorer-selected top-1 trajectory on `navtrain` and `navtest`, respectively. These diagnostic scores evaluate scorer ranking quality under fixed proposals and are not directly comparable to full-system CLOVER results.

Scorer backbone	Train score $\uparrow$	Test score $\uparrow$
DINOv2-Small	95.0	92.4
DINOv2-Base	95.3	92.6
DINOv3-Small	95.1	92.4
DINOv3-Base	95.1	92.7
Wan2.2-5B features	97.7	92.8

training cost in our closed-loop refinement pipeline. For these reasons, we keep DINOv2-Small as the deployed scorer in CLOVER and use the larger backbones only as diagnostic references.

The diagnostic results suggest two observations. First, scorer design has measurable impact even when the candidate pool is fixed, confirming that trajectory ranking is a meaningful bottleneck after proposal diversity has been improved. Second, stronger visual or video features can improve scorer performance, but the gain must be balanced against computational cost and overfitting risk. The large train-test gap of Wan2.2-5B indicates high representational capacity, but also suggests that using such a model as an online scorer inside the full CLOVER training loop would be impractical under our current training budget. We therefore release this scorer diagnostic protocol to support future studies on trajectory-level scoring, score calibration, and ranking architectures under a shared high-quality candidate pool.

G Seed Stability

The official NAVSIM evaluation is deterministic given a trained model and a fixed benchmark split. Therefore, the main source of stochastic variation in our reported results comes from training randomness, including parameter initialization, dataloader order, and stochastic optimization. To assess this effect, we repeat the main CLOVER training with three random seeds under the same architecture, data split, hyperparameters, and evaluation protocol.

Table 21 reports the seed-level variation. Across three runs, the final NAVSIM v1 PDMS differs by less than 0.02 points. This variation is below the one-decimal reporting precision used in the main benchmark tables and does not affect any conclusion or ranking trend. We therefore report the official single-run benchmark scores in the main paper and use this experiment only to verify run-to-run stability.

H Additional Proposal Diversity Metrics

We provide additional proposal-set statistics comparing the DrivoR baseline, Stage 1 diversity pretraining, and Stage 2 self-distillation refinement. All metrics are averaged over 12,146 common scenes, with 64 proposals per scene. The true PDMS scores are used only for analysis.

Table 21: **Seed stability of CLOVER.** All runs use the same architecture, training schedule, data split, and evaluation protocol. The observed variation is below the one-decimal reporting precision of the main benchmark tables.

Setting	Seed 1	Seed 2	Seed 3	Max-Min
NAVSIM v1 PDMS	94.488	94.502	94.492	< 0.02
NAVSIM v2 EPDMS	90.354	90.358	90.368	< 0.01
NAVSIM v2 EPDMS*	87.201	87.207	87.210	< 0.01

Table 22: **Top-K proposal quality under scorer ranking.** TopK-Oracle@K reports the best true PDMS score among the scorer-ranked top- K proposals. TopK-Mean@K reports the average true PDMS score within the scorer-ranked top- K proposals.

Metric	K	Baseline	Stage 1	Stage 2	Stage1 - Base	Stage2 - Stage1
TopK-Oracle@K	1	0.9369	0.9413	0.9443	+0.0043	+0.0031
TopK-Oracle@K	2	0.9461	0.9514	0.9517	+0.0054	+0.0002
TopK-Oracle@K	3	0.9512	0.9570	0.9554	+0.0058	-0.0017
TopK-Oracle@K	6	0.9603	0.9660	0.9625	+0.0057	-0.0035
TopK-Mean@K	1	0.9369	0.9413	0.9443	+0.0043	+0.0031
TopK-Mean@K	2	0.9353	0.9402	0.9435	+0.0049	+0.0033
TopK-Mean@K	3	0.9345	0.9383	0.9430	+0.0039	+0.0046
TopK-Mean@K	6	0.9320	0.9346	0.9416	+0.0026	+0.0070

Quality metrics. Selected PDMS is the true PDMS score of the trajectory selected by the learned scorer. Oracle@64 is the best true PDMS score among all 64 proposals. Mean PDMS over 64 measures the average quality of the full proposal set. Gap is the difference between Oracle@64 and Selected PDMS. Count(PDMS > 0.95), Count(PDMS > 0.90), and Count(PDMS < 0.50) summarize the proposal score distribution.

Diversity metrics. Pairwise ADE/FDE measure the average pairwise distance between generated proposals. Endpoint Std Radius and Endpoint Area measure the spread of final trajectory endpoints. Trajectory Effective Rank is computed by flattening each 8-step (x, y) trajectory into a vector, centering the proposal set, and computing the entropy-based effective rank of the singular-value spectrum. Endpoint Cluster Count@ r greedily counts endpoint clusters under radius r .

Quality-aware diversity. To avoid overestimating diversity due to low-quality outliers, we also report diversity after filtering proposals with PDMS ≥ 0.8 . Qualified Cluster Count@2m measures the number of distinct endpoint clusters among qualified proposals. Qualified Pairwise ADE/FDE measures diversity within qualified proposals. Top-6 Real Pairwise ADE/FDE measures diversity among the six proposals with the highest true PDMS scores in each scene.

Interpretation. The three stages exhibit distinct behavior. Stage 1 is an expansion stage: it increases the oracle upper bound and raw geometric diversity, but also introduces more low-score proposals. Stage 2 is a refinement stage: it preserves much of the expanded support while improving scorer-selected top-1, top- K mean quality, full-set mean quality, and low-score tail suppression. This supports the design of CLOVER as a two-stage process: first broaden the proposal space, then refine it toward high-value regions.

H.1 Additional Qualitative Comparisons of Proposal Diversity

Figure 5 presents additional qualitative comparisons between the DrivoR baseline and CLOVER on six more scenes, following the same visualization protocol as Figure 3 in the main text. For each scene, the left two panels show the DrivoR baseline and the right two panels show CLOVER after Stage-2 refinement. For each method, we visualize all 64 generated candidate trajectories in both the front-view image and bird’s-eye view. Candidate trajectories are colored by their true PDMS scores for analysis, while the human trajectory, the scorer-selected trajectory, and the oracle trajectory with the highest true PDMS are highlighted explicitly.

Table 23: **Full proposal-set quality statistics.** Stage 1 achieves the highest oracle upper bound but also introduces a lower-quality tail. Stage 2 refines the expanded proposal distribution and achieves the best selected score, smallest oracle-selection gap, highest proposal mean, and fewest low-score proposals.

Metric	Baseline	Stage 1	Stage 2	Stage1 - Base	Stage2 - Stage1	Stage2 - Base
Selected PDMS $\uparrow$	0.9369	0.9413	0.9443	+0.0043	+0.0031	+0.0074
Oracle@64 PDMS $\uparrow$	0.9933	0.9976	0.9939	+0.0042	-0.0037	+0.0005
Gap $\downarrow$	0.0564	0.0563	0.0495	-0.0001	-0.0068	-0.0069
Mean PDMS over 64 $\uparrow$	0.7972	0.7374	0.8277	-0.0597	+0.0903	+0.0306
Std PDMS over 64	0.2152	0.3013	0.2235	+0.0861	-0.0778	+0.0084
Count(PDMS > 0.95) $\uparrow$	30.94	24.78	31.97	-6.16	+7.19	+1.03
Count(PDMS > 0.90) $\uparrow$	40.24	32.13	39.90	-8.10	+7.77	-0.33
Count(PDMS < 0.50) $\downarrow$	9.05	12.21	6.83	+3.16	-5.39	-2.22

Table 24: **Full proposal diversity statistics.** Stage 1 produces the widest raw proposal spread, while Stage 2 slightly contracts this spread but remains substantially more diverse than the baseline. Higher is better for all metrics.

Metric	Baseline	Stage 1	Stage 2	Stage1 - Base	Stage2 - Stage1	Stage2 - Base
Pairwise ADE	1.8004	5.9724	5.1956	+4.1720	-0.7768	+3.3952
Pairwise FDE	4.4660	10.6973	8.2536	+6.2313	-2.4438	+3.7876
Endpoint Std Radius	3.9033	10.2638	7.9477	+6.3604	-2.3160	+4.0444
Endpoint Area	5.7837	46.1758	25.5615	+40.3921	-20.6143	+19.7777
Trajectory Effective Rank	1.1401	1.2362	1.2748	+0.0962	+0.0385	+0.1347

These additional examples consistently support the same conclusion as in the main text. The DrivoR baseline tends to concentrate proposals around a narrow single mode, often with limited lateral or longitudinal variation. In contrast, CLOVER produces a broader set of feasible candidates, covering richer within-lane offsets, progress patterns, and alternative motion branches. Importantly, this increased diversity is not merely caused by low-quality outliers: many diverse candidates remain high-scoring under the evaluator, and the scorer-selected trajectory typically remains close to the oracle high-score mode. Together with the quantitative results in Section 5.3, these visualizations further confirm that CLOVER improves proposal diversity while preserving strong proposal quality.

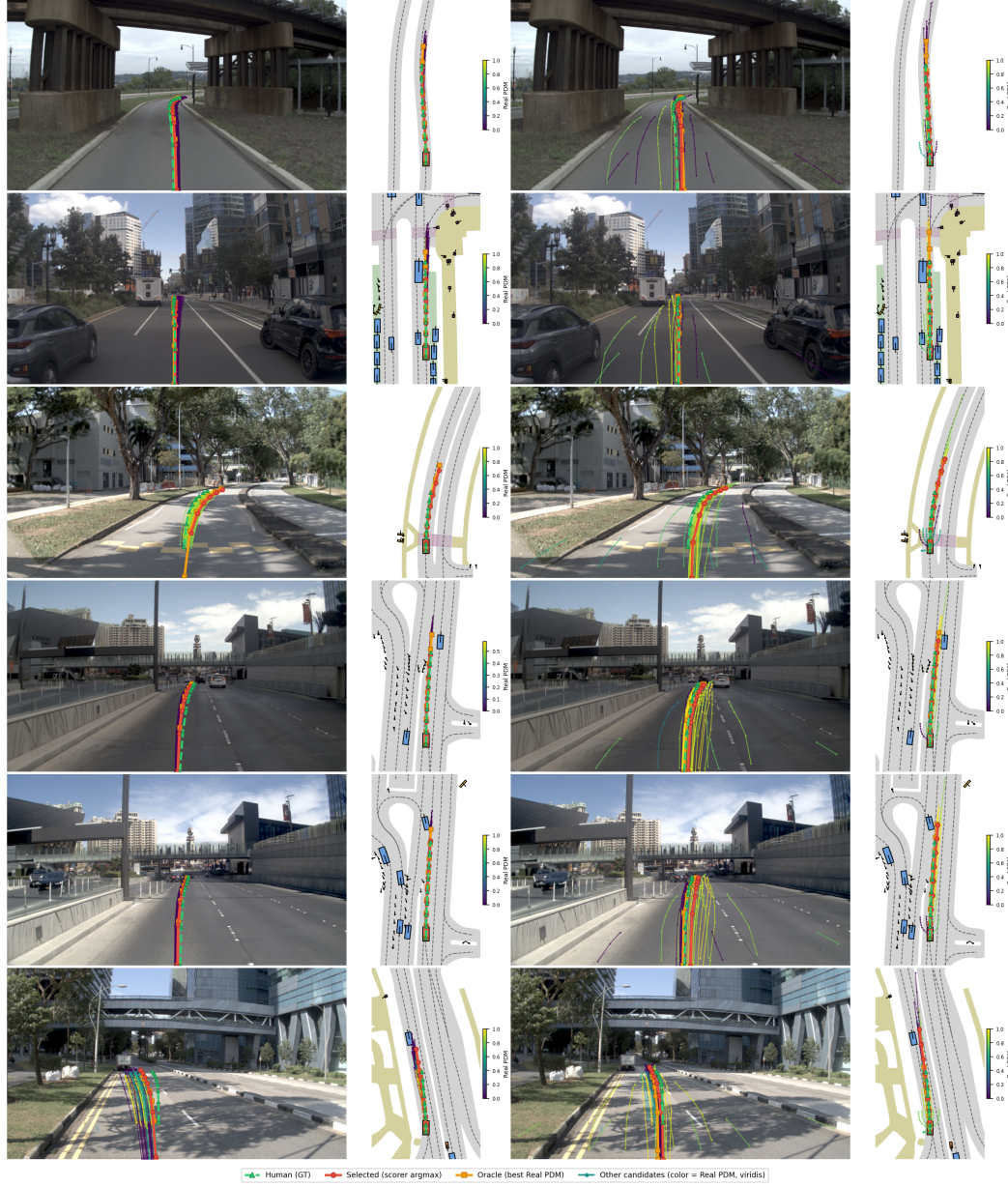


Figure 5: **Additional qualitative comparisons of proposal diversity on six scenes.** In each row, the left two panels show the DrivoR baseline and the right two panels show CLOVER after Stage-2 refinement. For each method, we visualize all 64 candidate trajectories in the front-view image and in bird’s-eye view. Candidate trajectories are colored by their true PDMS scores for analysis. Green triangles denote the human trajectory, red circles denote the scorer-selected trajectory, orange squares denote the oracle trajectory with the highest true PDMS, and the remaining candidates are colored by real PDMS using the viridis colormap. Across all six scenes, the baseline proposals are concentrated around a narrow mode, whereas CLOVER consistently covers a wider set of feasible and often high-scoring candidates.

Table 25: **Quality-aware proposal diversity.** Diversity is computed either after filtering proposals with $\text{PDMS} \geq 0.8$ or within the six highest-scoring proposals according to the true evaluator. This avoids attributing diversity gains only to low-quality outliers.

Metric	Baseline	Stage 1	Stage 2	Stage1 - Base	Stage2 - Stage1	Stage2 - Base
Qualified Count($\text{PDMS} \geq 0.8$)	50.16	43.97	50.44	-6.19	+6.47	+0.27
Qualified Cluster Count@2m	6.02	10.91	8.71	+4.90	-2.20	+2.70
Qualified Pairwise ADE	1.5576	3.7988	3.6727	+2.2412	-0.1261	+2.1151
Qualified Pairwise FDE	3.8024	6.8121	5.6674	+3.0097	-1.1447	+1.8650
Top-6 Real Pairwise ADE	0.9110	2.1111	1.7507	+1.2001	-0.3603	+0.8398
Top-6 Real Pairwise FDE	2.1955	3.5542	2.5910	+1.3587	-0.9633	+0.3954

Table 26: **Endpoint cluster count under different radii.** Endpoint clusters are computed by greedy clustering over final trajectory endpoints. Stage 1 produces the most endpoint branches, while Stage 2 remains consistently more diverse than the baseline at all radii.

Radius	Baseline	Stage 1	Stage 2
1m	16.77	32.77	23.20
2m	8.07	19.81	13.81
3m	5.29	14.65	9.88
4m	3.97	11.24	8.07